

FlowForge Complete System Manual & Line-By-Line Guide

This is the exhaustive documentation for FlowForge, detailing the architecture, providing an absolute beginner's breakdown, and providing line-by-line breakdowns of all core system files.

Part 1: Absolute Beginner's Core Concepts Guide

Part 4: Absolute Beginner's Core Concepts Guide

If you are brand new to full-stack web development, the technical terms used in the previous sections might seem overwhelming. This section breaks down the core concepts of "What" and "Why" assuming absolutely zero prior programming experience.

1. What is a Database? (And why PostgreSQL?)

Imagine you have a spreadsheet where you keep track of your users. You have a column for "Name", "Email", and "Password". A **Database** is essentially an extremely powerful, fast, and secure version of that spreadsheet.

PostgreSQL is the specific brand of database we are using. It is a "Relational Database", meaning it uses tables (like spreadsheet tabs) that relate to each other. For example, a User "relates" to a Workspace.

2. What is an ORM? And what is Prisma?

Databases don't speak Javascript or English; they speak a language called SQL (Structured Query Language). To ask the database for all users, you would normally have to write raw text: `SELECT * FROM users;`

Writing raw SQL inside JavaScript code gets messy and is prone to typos.

An **ORM (Object-Relational Mapper)** is a translator. It lets you write normal JavaScript code, and it translates it into SQL for you securely. **Prisma** is the specific ORM we use.

Instead of raw SQL, Prisma lets us write: `const users = await prisma.user.findMany();` Prisma translates that to SQL, asks the database, and returns the result as an easy-to-use JavaScript array.

3. What is a Schema? (`schema.prisma`)

Before Prisma can translate your code, it needs to know what your "spreadsheets" (tables) look like. The **Schema** is simply the "blueprint" of your database.

When you write `schema.prisma`, you are explicitly telling the database: *"I want a table called User. It must have a column called Email, and that column must contain text, and no two people can have the same email."*

If you don't define the schema, Prisma doesn't know what data is allowed to be saved.

4. What is an API? (Express.js)

Your React Frontend (the visual website) is not allowed to talk directly to your PostgreSQL database. If it could, hackers could use their browser to delete your massive database.

Instead, we put a "bouncing guard" in between the frontend and the database. This guard is the **Backend API**. **Express.js** is the tool we use to build this API guard.

When you try to log in, the Frontend doesn't check the database. It sends a "Request" to the API: *"Hey API, here is an email and password."* The API (Express) receives it, uses Prisma to check the database safely, and sends a "Response" back to the frontend: *"Yes, the password is correct."*

5. What is Authentication and JWT?

When you log in successfully, how does the API remember who you are when you click to a new page?

It uses a **JWT (JSON Web Token)**. Think of a JWT as a digital, unforgeable wristband you get at a concert.

1. You show your ID (Email/Password) to the API.
2. The API verifies it and hands you a wristband (a JWT string of random letters like `eyJhb...`).
3. Your browser saves this wristband.
4. From now on, whenever your frontend asks the API for your private workflows, it shows the wristband in the request header (`Authorization: Bearer eyJhb...`).
5. The API sees the valid wristband and gives you your data without making you log in again.

6. How does the Visual Canvas (React Flow) work?

React Flow is a library that handles the insane mathematics of dragging visual blocks around a screen.

At its core, React Flow only understands two simple Lists (Arrays):

1. **Nodes:** A list of blocks. Every block has an `id` (e.g., "node_1") and a position (X: 100, Y: 200).
2. **Edges:** A list of wires. Every wire has a `source` (e.g., "node_1") and a `target` (e.g., "node_2").

When you drag a block on the screen, React Flow is just constantly updating the X and Y coordinate numbers in the background. When you hit "Save", we take those two simple lists of numbers and text, convert them into **JSON** (which is just a way to save Javascript objects as plain text), and tell the API to save that text pattern into our Database. When you refresh the page, we read that text back, turn it into Lists, and React Flow immediately draws the blocks back exactly where their coordinates say they belong.

Part 2: Architecture & System Design

FlowForge — System Design & Architecture

This document provides a comprehensive overview of the FlowForge system design. Use this to understand the high-level architecture, data flows, and technical decisions made when building the application.

1. High-Level Architecture

FlowForge is a full-stack Next.js and Express.js application designed to provide a visual workflow automation experience (similar to heavily scaled systems like Make.com, n8n, or Zapier).

The system is broken down into three primary layers:

1. **Frontend (Next.js / React):** Handles the UI, the visual React Flow canvas, authentication views, and real-time dashboard updates.
2. **Backend API (Express.js):** A RESTful API that handles CRUD operations (Workflows, Executions, Credentials, Auth) and relies on PostgreSQL for persistence.
3. **Execution Engine (BullMQ + Redis):** An asynchronous background worker system that safely executes workflows node-by-node without blocking the main API thread.

System Diagram

```
graph TD
    subgraph Frontend "Next.js (Port 3000)"
        UI[React UI Component]
        Canvas[React Flow Canvas]
        Store[Zustand Store]
        SocketClient[Socket.io Client]
    end

    subgraph Backend "Express API (Port 4000)"
        API[Express Router]
        Auth[JWT Auth Middleware]
        Zod[Zod Validation]
        SocketServer[Socket.io Server]
    end

    subgraph Worker "Execution Engine"
        Queue[BullMQ Queue]
        Runner[Workflow Runner]
        Executor[Node Executors]
    end

    subgraph Infrastructure
        DB[(PostgreSQL)]
        Redis[(Redis Cache/Queue)]
    end

    %% Flow connections
    UI -->|REST API| API
    Canvas -->|State| Store
    Store -->|Save Data| API
    API -->|Read/Write| DB
    API -->|Push Job| Queue
    Queue -->|Consume Job| Runner
    Runner -->|Execute Step| Executor
    Executor -->|Fetch API/DB/Mail| External[External APIs]
    Runner -->|Write Status| DB
    Runner -->|Emit Event| SocketServer
    SocketServer -->|Real-time update| SocketClient
    SocketClient -->|Update UI| UI
```

2. Database Schema (Prisma)

The database is PostgreSQL, accessed via Prisma ORM for strong TypeScript typing.

- **User** : Stores user credentials, email, and hashed passwords.
- **Workspace** : A collaborative environment. A user owns a workspace, and workflows belong to workspaces.
- **Workflow** : The core entity. Stores the `nodesJson` and `edgesJson` which represent the visual canvas, along with status (`DRAFT` , `ACTIVE`).
- **Execution** : Created every time a workflow runs. Tracks overall status (`PENDING` , `RUNNING` , `SUCCESS` , `FAILED`) and duration.
- **NodeExecution** : A granular log of a single node's run inside a specific `Execution` . Stores the output JSON or the error message if the node failed.

Relational Flow

User --(1:1)--> Workspace --(1:N)--> Workflow --(1:N)--> Execution --(1:N)--> NodeExecution

3. The Execution Engine (How Workflows Actually Run)

When a user clicks "Run Now", the following lifecycle occurs:

1. **Trigger**: Frontend sends `POST /api/workflows/:id/execute` .
 2. **Database Record**: Backend creates a new `Execution` record in Postgres marked as `PENDING` .
 3. **Queueing**: Backend adds a job to the Redis `workflow-queue` using BullMQ.
 4. **Processing**: The Worker (running in the backend) picks up the job.
 5. **DAG Sorting**: The worker parses `nodesJson` and `edgesJson` . It creates a Directed Acyclic Graph (DAG) and sorts nodes topologically to determine execution order.
 6. **Execution Loop**: The worker loops through the sorted nodes:
 - Evaluates the Node type (e.g., `START` , `HTTP_REQUEST` , `EMAIL`).
 - Calls the correct `nodeExecutor.ts` function.
 - Saves the result payload to Postgres as a `NodeExecution` .
 - Emits a Socket.IO event to the frontend showing live progress.
 7. **Completion**: Once all nodes succeed, the `Execution` is marked `COMPLETED` . If a node throws an error, execution stops, and it is marked `FAILED` .
-

4. Security & Validation

- **Authentication**: JWT (JSON Web Tokens) are generated upon login. The frontend stores it in `localStorage` and sends it as a `Bearer` token in the `Authorization` header for all requests.
 - **Zod Validation**: Every backend endpoint passes through `validateRequest.ts` . Zod ensures that incoming data matches strict schemas (e.g., stopping users from creating workflows without names or passing invalid JSON).
 - **CORS & Rate Limiting**: The backend employs Helmet, CORS, and Express-Rate-Limit to prevent excessive requests and unauthorized domain access.
-

5. Technology Stack Summary

- **Frontend**: Next.js (App Router), React, Tailwind CSS, Framer Motion, React Flow (for canvas), Zustand (global state).

- **Backend:** Node.js, Express.js, Prisma ORM, JSON Web Tokens (JWT), Zod.
- **Execution & Real-time:** BullMQ, Redis, Socket.IO.
- **Database:** PostgreSQL.

Part 3: Implementation & Codebase Walkthrough

FlowForge — Codebase Walkthrough & Line-by-Line Guide

This document breaks down the FlowForge codebase so you can understand exactly what each file does, why it exists, and how the pieces connect. It serves as your mental map of the project.

1. Directory Structure

The project is structured entirely as a **Monorepo** using npm workspaces. This means the frontend and backend live side-by-side.

```
/Users/tanmoysaha/development/FlowForge/
├── packages/
│   ├── web/           # The Next.js Frontend Application
│   └── server/        # The Express.js Backend API & Worker Engine
```

2. Packages / Server (The Backend)

The backend is responsible for data persistence, authentication, API routes, and running the background execution engine.

Core Configuration Files

- **packages/server/.env** : Stores secret variables like the `DATABASE_URL` , `REDIS_URL` , `JWT_SECRET` , and API keys (SMTP credentials for emails, OpenAI keys). **Never commit this to Github.**
- **packages/server/src/index.ts** : The absolute entry point of the backend. It instantiates the Express server, connects socket.io, applies security middleware (cors, helmet), registers the API routes, and starts the BullMQ background worker.
- **packages/server/src/prisma/schema.prisma** : The single source of truth for the database structure. Defines `User` , `Workspace` , `Workflow` , `Execution` , and `NodeExecution` tables. Running `npx prisma migrate` translates this file into real SQL tables.

The API Routes (`src/routes/`)

These files define the HTTP endpoints the frontend talks to.

- **auth.ts** : Contains `/api/auth/register` and `/api/auth/login` . It hashes passwords with `bcryptjs` and converts correct credentials into JWT tokens.
- **workflows.ts** : The CRUD endpoints for saving and loading the canvas. Notably, `PUT /api/workflows/:id` receives `nodesJson` and `edgesJson` from the frontend and stores it exactly as drawn.

- **executions.ts** : Contains the route that triggers a workflow. It accepts a `POST` request, writes a `PENDING` execution to the database, and adds a job to the BullMQ Redis queue.

The Brain / Execution Engine (`src/services/`)

This is the most complex logic in the system.

- **workflowExecutor.ts** : The BullMQ worker. It listens for new jobs in Redis. When a job arrives, it grabs the respective workflow from PostgreSQL, performs a Topological Sort to figure out which nodes to run first, and iterates through them.
- **nodeExecutor.ts** : Contains the actual code that runs when a specific node is processed. For example, if it hits an `EMAIL` node, `nodeExecutor.ts` dynamically runs the `nodemailer` code to send an email using the SMTP credentials from your `.env` file. If it hits an `HTTP_REQUEST` node, it executes an actual JS `fetch()` call.

Safety & Middleware (`src/middleware/`)

- **auth.ts (Middleware)**: Intercepts all API requests (except login/register), looks for the `Authorization: Bearer <token>` header, verifies the token using `jsonwebtoken`, and attaches the `userId` to the request. If missing or invalid, it rejects the request with a 401 Unauthorized.
- **validate.ts & src/schemas/** : Uses `Zod` to establish strict schemas. For example, it guarantees an incoming workflow payload has a `name` string and a valid `workspaceId` before Prisma even attempts to query the database.

3. Packages / Web (The Frontend)

The frontend is built on Next.js 14 inside the `app/` router directory layout. It uses React Flow for the visual canvas.

The Layouts

- **app/layout.tsx** : The master root file mapping the `<html>` tag. It wraps the entire app in `GlobalErrorBoundary` and the `<Toaster>` so popup notifications work everywhere.
- **app/(auth)/layout.tsx** : The layout specifically for login/register pages. Shows a split-screen design.
- **app/(dashboard)/layout.tsx** : The dashboard layout. Contains the `Sidebar.tsx` navigation and checks if the user is authenticated; if not, forcefully boots them to `/login`.

Key Pages

- **app/(auth)/login/page.tsx** : Uses `react-hook-form` to collect emails and passwords securely, validates them synchronously on the client via `Zod`, and sends the request to the `auth.ts` backend route.
- **app/(dashboard)/page.tsx** : The Dashboard view displaying metrics and a quick-action list of your workflows.
- **app/(dashboard)/workflows/page.tsx** : The dedicated Workflow management grid where you can create new automation flows.
- **app/(dashboard)/editor/[id]/page.tsx** : The visual React Flow editor. This is the masterpiece of the UI. It pulls the loaded workflow JSON into the Zustand `workflowStore.ts`.

React Flow Canvas (`components/canvas/`)

- **Canvas.tsx** : The wrapper around the `<ReactFlow>` provider. Renders the grid backdrop and the actual draggable nodes. Integrates heavily with the Zustand store to handle drag drops.
- **NodePalette.tsx** : The left sidebar containing all 10 available blocks. Contains logic that allows users to drag these HTML elements and drop them into the canvas to generate a node.
- **ConfigPanel.tsx** : The sliding right sidebar. When you click a node on the canvas, it sets that node as "selected". The Config Panel reads the selected node type and dynamically renders an input form customized for that node (e.g., showing a "Webhook URL" field for webhook nodes, or a "To / Subject" field for Email nodes).

State Management (stores/)

Because React Flow state is massive, we use Zustand instead of standard React Context to prevent unnecessary re-renders.

- **workflowStore.ts** : Holds the `nodes` and `edges` arrays. This store provides functions like `addNode` , `onNodesChange` , and `connectEdge` that React Flow requires to update its UI when the user interacts with the canvas.
- **authStore.ts** : Holds the logged-in user object, their `token` , and their `activeWorkspaceId` . It synchronizes this data with browser `localStorage` so a user stays logged in after a page refresh.

API Client (lib/api.ts)

- **api.ts** : A centralized `fetch` wrapper. Instead of writing raw `fetch('http://localhost:4000/api/...')` in components, all files call `workflowApi.get()` or `authApi.login()` . The client automatically hooks into `localStorage` to append the JWT Bearer token to every request securely.

4. Summary of "Why" We Wrote This

If asked in an interview why the architecture looks like this:

1. **Next.js & React Flow**: For building a complex, highly interactive, yet performant user interface capable of heavy DOM manipulation.
2. **Node.js, Express, Prisma**: Because TypeScript provides full-stack type safety. We can define our schemas in Prisma, validate in Zod, and share those types directly with the Next.js frontend to ensure no data mismatches.
3. **BullMQ / Redis**: Heavy tasks (like sending emails or pinging third-party APIs) take time. If the Express REST API ran these synchronously, the frontend UI would freeze waiting for a response. By offloading it to BullMQ, the frontend gets an instant 200 OK "Started" message, and Socket.io streams the progress asynchronously.

Part 4: From-Scratch Build Tutorial

How to Build FlowForge From Scratch

The Ultimate Step-by-Step Guide

This guide breaks down exactly how to recreate the FlowForge project from an empty folder. It explains *what* code to write and, more importantly, *why* every line is written.

Because FlowForge is a massive full-stack application, this guide focuses on the critical architectural pillars.

Step 1: Initializing the Monorepo

We use a Monorepo (npm workspaces) so our frontend and backend can live together but have separate `package.json` files.

1. Create the root folder and configure the workspace:

```
mkdir flowforge
cd flowforge
npm init -y
```

2. Edit the root `package.json` to define workspaces:

```
{
  "name": "flowforge-root",
  "private": true,
  "workspaces": [
    "packages/*"
  ]
}
```

Why? The `"workspaces"` array tells `npm` that any folder inside `packages/` is its own distinct project, but they can share `node_modules` at the root.

3. Create the two main packages:

```
mkdir packages
cd packages
npx create-next-app@latest web # (Choose Yes to TypeScript, Tailwind, App Router)
mkdir server && cd server && npm init -y
```

Step 2: Setting up the Database (Prisma)

Before writing the API, we need a database schema. We use Prisma because it generates TypeScript types automatically based on our database structure.

1. Inside `packages/server`, install Prisma:

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

2. Open `packages/server/prisma/schema.prisma` and write the Schema:

```
// This block tells Prisma to generate a standard JavaScript/TypeScript client
generator client {
  provider = "prisma-client-js"
}
```

```
// This block tells Prisma we are connecting to a Postgres database via a URL in the
// .env file
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// The User Model
model User {
  id          String      @id @default(cuid()) // CUIDs are unique random strings
  name        String?     // ? means optional
  email       String      @unique              // No two users can have the same
  email
  passwordHash String?     // Optional for Google Auth users
  createdAt   DateTime     @default(now())

  // Relations: A user can own multiple workspaces
  workspaces  Workspace[] @relation("WorkspaceOwner")
}

// The Workflow Model (The Core Entity)
model Workflow {
  id          String      @id @default(cuid())
  name        String
  nodesJson   Json        @default("[]") // Stores the React Flow canvas blocks
  edgesJson   Json        @default("[]") // Stores the lines connecting the blocks
  status      String      @default("DRAFT")

  // Relation: belongs to a specific workspace
  workspaceId String
  workspace    Workspace @relation(fields: [workspaceId], references: [id])
}
```

Why? We use `Json` fields for `nodesJson` and `edgesJson` instead of relational tables because React Flow canvas data is wildly dynamic and deeply nested. It's much faster to query a whole JSON blob and pass it directly to the frontend than it is to join 100 individual "node" rows in SQL.

Step 3: The Backend Entry Point (`server/src/index.ts`)

This is the file that boots up the entire backend.

```
import express from 'express';
import cors from 'cors';

// 1. Initialize Express
const app = express();

// 2. Apply Middleware
// CORS allows our frontend running on port 3000 to talk to this backend on port
// 4000
app.use(cors({ origin: 'http://localhost:3000', credentials: true }));
```

```
// JSON parses incoming API payloads so we can read `req.body`
app.use(express.json());

// 3. Define a basic route
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});

// 4. Start listening on a port
app.listen(4000, () => {
  console.log('Server is running on port 4000');
});
```

Why? Express is the foundation. Every API request (like saving a workflow) goes through `app.use(express.json())` so it becomes a JavaScript object, and then matches a specific route logic.

Step 4: The Frontend Canvas

(`web/components/canvas/Canvas.tsx`)

This is where the magic visual builder happens. We use a library called `reactflow`.

```
'use client'; // Required in Next.js 14 for components that use browser features
               (like dragging)

import ReactFlow, { Background, Controls } from 'reactflow';
import 'reactflow/dist/style.css'; // The default styling for nodes and wires

export default function Canvas() {
  // 1. Define nodes (the blocks)
  const initialNodes = [
    { id: '1', position: { x: 100, y: 100 }, data: { label: 'Start Here' } },
    { id: '2', position: { x: 100, y: 200 }, data: { label: 'Send Email' } }
  ];

  // 2. Define edges (the connection wires)
  const initialEdges = [
    { id: 'e1-2', source: '1', target: '2' } // Connects node 1 to node 2
  ];

  return (
    <div style={{ width: '100vw', height: '100vh' }}>
      <ReactFlow nodes={initialNodes} edges={initialEdges}>
        {/* Adds grid dots in the background */}
        <Background />
        {/* Adds zoom in/out buttons */}
        <Controls />
      </ReactFlow>
    </div>
  );
}
```

```
    );  
  }  
}
```

Why? `ReactFlow` does all the mathematical heavy lifting of calculating canvas coordinates. All we have to do is provide an Array of `nodes` (with X and Y coordinates) and an Array of `edges` indicating what is connected to what. When a user creates a real workflow, we are simply mutating these two Arrays and saving them to Prisma's `nodesJson` field.

Step 5: Connecting Frontend to Backend using API wrappers (`web/lib/api.ts`)

Instead of writing `fetch('http://localhost:4000/api/...')` repeatedly inside our components, we create a centralized wrapper.

```
// Look for the token in LocalStorage  
function getToken() {  
  return localStorage.getItem('flowforge-token');  
}  
  
// Global fetch wrapper function  
async function apiRequest<T>(endpoint: string, options: RequestInit = {}) {  
  const token = getToken();  
  
  // Every request gets the Authorization header automatically  
  const headers = {  
    'Content-Type': 'application/json',  
    ...(token && { Authorization: `Bearer ${token}` } ),  
  };  
  
  const response = await fetch(`http://localhost:4000${endpoint}`, {  
    ...options,  
    headers,  
  });  
  
  return await response.json();  
}  
  
// Typed API methods  
export const workflowApi = {  
  // Notice we don't have to write fetch, headers, or token logic here anymore  
  get: (id: string) => apiRequest(`/api/workflows/${id}`),  
  delete: (id: string) => apiRequest(`/api/workflows/${id}`, { method: 'DELETE' }),  
};
```

Why? This is crucial for codebase maintainability. If we ever deploy to real servers and the URL changes, we only change it here. It also ensures we never randomly forget to attach the JWT Auth Token to a request.

Step 6: How the Execution Engine actually works

(server/src/services/nodeExecutor.ts)

When a workflow runs, the backend reads the array of nodes from the database and runs them one by one.

Here is what the exact execution logic for a "Send Email" node looks like:

```
import nodemailer from 'nodemailer';

// This function receives the exact data block the user typed into the config panel
// UI
export async function executeEmailNode(nodeData: any) {

  // 1. Ensure required fields were provided by the user
  if (!nodeData.to || !nodeData.subject || !nodeData.body) {
    throw new Error('Missing required email fields (to, subject, body)');
  }

  // 2. Instantiate Nodemailer using our system environment variables
  const transporter = nodemailer.createTransport({
    host: process.env.SMTP_HOST,
    port: parseInt(process.env.SMTP_PORT || '587'),
    auth: {
      user: process.env.SMTP_USER,
      pass: process.env.SMTP_PASS, // Example: Google App Password
    },
  });

  // 3. Actually send the email over the network
  const info = await transporter.sendMail({
    from: process.env.SMTP_FROM,
    to: nodeData.to,           // Inject the user's config
    subject: nodeData.subject, // Inject the user's config
    text: nodeData.body,       // Inject the user's config
  });

  // 4. Return what happened so it can be saved in the Execution History logs
  return {
    messageId: info.messageId,
    status: 'Sent Successfully'
  };
}
```

Why? The frontend UI's only job is to collect strings ("Who is this email to?"). The backend's job is to read those strings from PostgreSQL and execute real server code (Nodemailer, native HTTP Fetches, raw SQL connectors).

End of Part 1.

This document outlines exactly how the core foundation connects. A true "line by line" copy-paste of a 60-file application is beyond standard document size, but the patterns demonstrated above (Prisma -> Express -> React Flow -> Executor) are repeated universally throughout the entire FlowForge application.

Part 5: Line-by-Line Code Analysis

The following section contains the raw source code for the most critical components of the system, followed by detailed line-by-line explanations.

File: `packages/server/src/prisma/schema.prisma`

Source Code

```
// =====  
// FlowForge – Prisma Database Schema  
// =====  
// This file defines ALL the database tables (called "models" in Prisma).  
// Think of each model as a table in the database.  
// Relations are like links between tables (e.g., a User has many Workflows).  
  
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}  
  
// ---- ENUMS ----  
// Enums are like predefined options (dropdown values)  
  
enum WorkspaceRole {  
  OWNER  
  ADMIN  
  MEMBER  
  VIEWER  
}  
  
enum WorkflowStatus {  
  DRAFT  
  ACTIVE  
  PAUSED  
  ARCHIVED  
}  
  
enum TriggerType {  
  MANUAL  
  CRON  
  WEBHOOK
```

```

}

enum ExecutionStatus {
    PENDING
    RUNNING
    COMPLETED
    FAILED
    CANCELLED
}

enum NodeExecutionStatus {
    PENDING
    RUNNING
    SUCCESS
    FAILED
    SKIPPED
}

enum ExecutionTrigger {
    MANUAL
    CRON
    WEBHOOK
}

enum CredentialType {
    API_KEY
    OAUTH
    BASIC_AUTH
    CUSTOM
}

// ---- MODELS ----

/// A user who can log in and create workflows
model User {
    id          String    @id @default(cuid())
    email       String    @unique
    name        String
    passwordHash String?  // Nullable for OAuth-only users
    googleId    String?   @unique // Store Google OAuth identifier
    avatarUrl   String?
    createdAt   DateTime  @default(now())
    updatedAt   DateTime  @updatedAt

    // Relations – a user can own workspaces, be a member of workspaces, etc.
    ownedWorkspaces    Workspace[]
    workspaceMemberships WorkspaceMember[]
    createdWorkflows   Workflow[]
    createdCredentials Credential[]

    @@map("users") // Table name in the database
}

```

```

/// A workspace is like a team/organization (multi-tenant)
model Workspace {
  id          String    @id @default(cuid())
  name        String
  slug        String    @unique
  createdAt   DateTime @default(now())

  // Relations
  ownerId String
  owner  User  @relation(fields: [ownerId], references: [id])

  members    WorkspaceMember[]
  workflows  Workflow[]
  credentials Credential[]

  @@map("workspaces")
}

/// Links users to workspaces with a specific role
model WorkspaceMember {
  id      String    @id @default(cuid())
  role    WorkspaceRole @default(MEMBER)

  workspaceId String
  workspace   Workspace @relation(fields: [workspaceId], references: [id], onDelete: Cascade)

  userId String
  user   User  @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([workspaceId, userId]) // A user can only be in a workspace once
  @@map("workspace_members")
}

/// A workflow is the main entity – it contains the DAG (nodes + edges)
model Workflow {
  id          String    @id @default(cuid())
  name        String
  description String?
  status      WorkflowStatus @default(DRAFT)

  triggerType  TriggerType @default(MANUAL)
  cronExpression String?    // e.g., "0 9 * * *" = every day at 9am
  webhookId    String?     @unique

  // The DAG is stored as JSON – React Flow nodes and edges arrays
  nodesJson Json @default("[]")
  edgesJson Json @default("[]")

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

```

```

// Relations
workspaceId String
workspace Workspace @relation(fields: [workspaceId], references: [id], onDelete:
Cascade)

createdById String
createdBy User @relation(fields: [createdById], references: [id])

executions Execution[]
webhook Webhook?

@@map("workflows")
}

/// An execution is one run of a workflow
model Execution {
  id String @id @default(cuid())
  status ExecutionStatus @default(PENDING)
  triggeredBy ExecutionTrigger

  startedAt DateTime @default(now())
  completedAt DateTime?
  duration Int? // Duration in milliseconds

  errorMessage String?

  // Relations
  workflowId String
  workflow Workflow @relation(fields: [workflowId], references: [id], onDelete:
Cascade)

  nodeExecutions NodeExecution[]

  @@map("executions")
}

/// Tracks the execution status of each individual node in a workflow run
model NodeExecution {
  id String @id @default(cuid())
  nodeId String // Matches React Flow node id
  nodeType String
  nodeName String
  status NodeExecutionStatus @default(PENDING)

  inputData Json? // Data received from parent nodes
  outputData Json? // Data this node produced
  logs String? @db.Text // Accumulated log text

  startedAt DateTime?
  completedAt DateTime?
  duration Int? // Duration in milliseconds

```

```

    errorMessage String?
    retryCount    Int      @default(0)

    // Relations
    executionId String
    execution    Execution @relation(fields: [executionId], references: [id], onDelete:
Cascade)

    @@map("node_executions")
}

/// Stores encrypted credentials (API keys, passwords, etc.)
model Credential {
    id          String      @id @default(cuid())
    name        String
    type        CredentialType
    encryptedData String    @db.Text // AES-256 encrypted JSON

    createdAt DateTime @default(now())

    // Relations
    workspaceId String
    workspace    Workspace @relation(fields: [workspaceId], references: [id], onDelete:
Cascade)

    createdById String
    createdBy    User      @relation(fields: [createdById], references: [id])

    @@map("credentials")
}

/// Webhook endpoints for triggering workflows via HTTP
model Webhook {
    id          String @id @default(cuid())
    path        String @unique // Unique slug like "abc123"
    secret      String  // For verifying webhook signatures
    isActive    Boolean @default(true)
    lastTriggeredAt DateTime?

    // Relations
    workflowId String @unique
    workflow    Workflow @relation(fields: [workflowId], references: [id], onDelete:
Cascade)

    @@map("webhooks")
}

```

Line-by-Line Breakdown

- **Line 8:** generator client {

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 9:** `provider = "prisma-client-js"`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 10:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 12:** `datasource db {`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 13:** `provider = "postgresql"`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 14:** `url = env("DATABASE_URL")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 15:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 20:** `enum WorkspaceRole {`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 21:** `OWNER`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 22:** `ADMIN`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 23:** `MEMBER`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 24:** `VIEWER`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 25:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 27:** `enum WorkflowStatus {`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 28:** `DRAFT`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 29:** `ACTIVE`

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 30:** PAUSED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 31:** ARCHIVED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 32:** }
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 34:** enum TriggerType {
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 35:** MANUAL
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 36:** CRON
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 37:** WEBHOOK
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 38:** }
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 40:** enum ExecutionStatus {
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 41:** PENDING
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 42:** RUNNING
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 43:** COMPLETED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 44:** FAILED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 45:** CANCELLED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 46:** }

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 48:** enum NodeExecutionStatus {
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 49:** PENDING
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 50:** RUNNING
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 51:** SUCCESS
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 52:** FAILED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 53:** SKIPPED
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 54:** }
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 56:** enum ExecutionTrigger {
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 57:** MANUAL
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 58:** CRON
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 59:** WEBHOOK
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 60:** }
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 62:** enum CredentialType {
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 63:** API_KEY
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 64:** OAUTH

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 65:** `BASIC_AUTH`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 66:** `CUSTOM`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 67:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 72:** `model User {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 73:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 74:** `email String @unique`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 75:** `name String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 76:** `passwordHash String? // Nullable for OAuth-only users`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 77:** `googleId String? @unique // Store Google OAuth identifier`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 78:** `avatarUrl String?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 79:** `createdAt DateTime @default(now())`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 80:** `updatedAt DateTime @updatedAt`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 83:** `ownedWorkspaces Workspace[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 84:** `workspaceMemberships WorkspaceMember[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 85:** `createdWorkflows Workflow[]`

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 86:** `createdCredentials Credential[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 88:** `@@map("users") // Table name in the database`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 89:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 92:** `model Workspace {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 93:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 94:** `name String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 95:** `slug String @unique`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 96:** `createdAt DateTime @default(now())`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 99:** `ownerId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 100:** `owner User @relation(fields: [ownerId], references: [id])`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 102:** `members WorkspaceMember[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 103:** `workflows Workflow[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 104:** `credentials Credential[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 106:** `@@map("workspaces")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 107:** `}`

- *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 110:** `model WorkspaceMember {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 111:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 112:** `role WorkspaceRole @default(MEMBER)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 114:** `workspaceId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 115:** `workspace Workspace @relation(fields: [workspaceId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 117:** `userId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 118:** `user User @relation(fields: [userId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 120:** `@@unique([workspaceId, userId]) // A user can only be in a workspace once`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 121:** `@@map("workspace_members")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 122:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 125:** `model Workflow {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 126:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 127:** `name String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 128:** `description String?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.

- **Line 129:** `status WorkflowStatus @default(DRAFT)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 131:** `triggerType TriggerType @default(MANUAL)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 132:** `cronExpression String? // e.g., "0 9 * * *" = every day at 9am`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 133:** `webhookId String? @unique`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 136:** `nodesJson Json @default("[]")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 137:** `edgesJson Json @default("[]")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 139:** `createdAt DateTime @default(now())`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 140:** `updatedAt DateTime @updatedAt`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 143:** `workspaceId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 144:** `workspace Workspace @relation(fields: [workspaceId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 146:** `createdById String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 147:** `createdBy User @relation(fields: [createdById], references: [id])`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 149:** `executions Execution[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 150:** `webhook Webhook?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 152:** `@@map("workflows")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.

the database table.

- **Line 153:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 156:** `model Execution {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 157:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 158:** `status ExecutionStatus @default(PENDING)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 159:** `triggeredBy ExecutionTrigger`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 161:** `startedAt DateTime @default(now())`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 162:** `completedAt DateTime?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 163:** `duration Int? // Duration in milliseconds`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 165:** `errorMessage String?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 168:** `workflowId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 169:** `workflow Workflow @relation(fields: [workflowId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 171:** `nodeExecutions NodeExecution[]`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 173:** `@@map("executions")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 174:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 177:** `model NodeExecution {`
 - *Explanation:* Defines a database table inside PostgreSQL.

- **Line 178:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 179:** `nodeId String // Matches React Flow node id`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 180:** `nodeType String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 181:** `nodeName String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 182:** `status NodeExecutionStatus @default(PENDING)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 184:** `inputData Json? // Data received from parent nodes`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 185:** `outputData Json? // Data this node produced`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 186:** `logs String? @db.Text // Accumulated log text`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 188:** `startedAt DateTime?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 189:** `completedAt DateTime?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 190:** `duration Int? // Duration in milliseconds`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 192:** `errorMessage String?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 193:** `retryCount Int @default(0)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 196:** `executionId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 197:** `execution Execution @relation(fields: [executionId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in

the database table.

- **Line 199:** `@@map("node_executions")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 200:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 203:** `model Credential {`
 - *Explanation:* Defines a database table inside PostgreSQL.
- **Line 204:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 205:** `name String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 206:** `type CredentialType`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 207:** `encryptedData String @db.Text // AES-256 encrypted JSON`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 209:** `createdAt DateTime @default(now())`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 212:** `workspaceId String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 213:** `workspace Workspace @relation(fields: [workspaceId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 215:** `createdById String`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 216:** `createdBy User @relation(fields: [createdById], references: [id])`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 218:** `@@map("credentials")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 219:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 222:** `model Webhook {`
 - *Explanation:* Defines a database table inside PostgreSQL.

- **Line 223:** `id String @id @default(cuid())`
 - *Explanation:* Sets this column as the primary key of the table (the unique identifier for a row).
- **Line 224:** `path String @unique // Unique slug like "abc123"`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 225:** `secret String // For verifying webhook signatures`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 226:** `isActive Boolean @default(true)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 227:** `lastTriggeredAt DateTime?`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 230:** `workflowId String @unique`
 - *Explanation:* Enforces a rule that no two rows can have the exact same value for this column.
- **Line 231:** `workflow Workflow @relation(fields: [workflowId], references: [id], onDelete: Cascade)`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 233:** `@@map("webhooks")`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.
- **Line 234:** `}`
 - *Explanation:* Defines a column and its data type (String, Boolean, Json, DateTime, etc.) in the database table.

File: `packages/server/src/services/nodeExecutor.ts`

Source Code

```
// =====
// FlowForge – Node Executor Service
// =====
// Real implementations for every node type.
// Each handler performs the actual action –
// no more simulations.

import nodemailer from 'nodemailer';
import { ChatGroq } from '@langchain/groq';
import { HumanMessage, SystemMessage } from '@langchain/core/messages';
import { logger } from '../utils/logger';

export interface NodeExecutionInput {
  nodeId: string;
```

```

nodeType: string;
label: string;
config: Record<string, unknown>;
previousOutput: unknown;
}

export interface NodeExecutionResult {
  success: boolean;
  output: unknown;
  error?: string;
  duration: number;
}

// =====
// Main Dispatch Function
// =====
export async function executeNode(input: NodeExecutionInput):
Promise<NodeExecutionResult> {
  const startTime = Date.now();

  try {
    let output: unknown;

    switch (input.nodeType) {
      case 'webhook-trigger':
        output = await executeWebhookTrigger(input);
        break;
      case 'http-request':
        output = await executeHttpRequest(input);
        break;
      case 'email':
        output = await executeEmail(input);
        break;
      case 'db-query':
        output = await executeDbQuery(input);
        break;
      case 'ai-llm':
        output = await executeAiLlm(input);
        break;
      case 'delay':
        output = await executeDelay(input);
        break;
      case 'condition':
        output = await executeCondition(input);
        break;
      case 'data-transform':
        output = await executeDataTransform(input);
        break;
      case 'code-execute':
        output = await executeCodeExecute(input);
        break;
      case 'slack-notify':

```

```

        output = await executeSlackNotify(input);
        break;
    default:
        throw new Error(`Unknown node type: ${input.nodeType}`);
    }

    return {
        success: true,
        output,
        duration: Date.now() - startTime,
    };
} catch (error) {
    const duration = Date.now() - startTime;
    const errMsg = error instanceof Error ? error.message : 'Unknown error';
    logger.error(`❌ Node "${input.label}" failed: ${errMsg}`);
    return {
        success: false,
        output: null,
        error: errMsg,
        duration,
    };
}
}

// =====
// Individual Node Handlers
// =====

// ---- Webhook Trigger ----
// Pass-through: the actual trigger data comes from the HTTP request that fired the
// workflow.
async function executeWebhookTrigger(input: NodeExecutionInput): Promise<unknown> {
    logger.info(`🔗 [${input.label}] Webhook trigger activated`);
    return {
        method: input.config.method || 'POST',
        path: input.config.path || '/webhook',
        body: input.previousOutput || { message: 'Webhook triggered' },
        headers: { 'content-type': 'application/json' },
        timestamp: new Date().toISOString(),
    };
}

// ---- HTTP Request ----
// Makes a REAL HTTP request to the configured URL.
async function executeHttpRequest(input: NodeExecutionInput): Promise<unknown> {
    const method = ((input.config.method as string) || 'GET').toUpperCase();
    const url = (input.config.url as string) || '';
    const customHeaders = (input.config.headers as Record<string, string>) || {};
    const bodyTemplate = input.config.body as string | undefined;

    if (!url) {
        throw new Error('HTTP Request node is missing a URL. Please configure the URL in

```

```

the node settings.');
```

```

}
```

```

logger.info(`🌐 [${input.label}] ${method} → ${url}`);
```

```

// Interpolate {input} with previous node output
```

```

let body: string | undefined;
```

```

if (['POST', 'PUT', 'PATCH'].includes(method) && bodyTemplate) {
```

```

    const inputStr = typeof input.previousOutput === 'object'
```

```
        ? JSON.stringify(input.previousOutput)
```

```
        : String(input.previousOutput || '');
```

```
    body = bodyTemplate.replace(/\{input\}/g, inputStr);
```

```

}
```

```

const response = await fetch(url, {
```

```
    method,
```

```
    headers: {
```

```
        'Content-Type': 'application/json',
```

```
        ...customHeaders,
```

```
    },
```

```
    ...(body ? { body } : {}),
```

```
    signal: AbortSignal.timeout(30000),
```

```
});
```

```

let data: unknown;
```

```

const contentType = response.headers.get('content-type') || '';
```

```

if (contentType.includes('application/json')) {
```

```
    data = await response.json();
```

```

} else {
```

```
    data = await response.text();
```

```

}
```

```

logger.info(`🌐 [${input.label}] Response: HTTP ${response.status}`);
```

```

if (!response.ok) {
```

```
    throw new Error(`HTTP ${response.status} ${response.statusText} from ${url}`);
```

```

}
```

```

return {
```

```
    status: response.status,
```

```
    statusText: response.statusText,
```

```
    url,
```

```
    method,
```

```
    data,
```

```

};
```

```

}
```

```

// ---- Email ----
```

```

// Sends a REAL email using Nodemailer + SMTP credentials from .env.
```

```

// Supports Gmail App Passwords (SMTP_USER + SMTP_PASS).
```

```

async function executeEmail(input: NodeExecutionInput): Promise<unknown> {
```

```
    const to = (input.config.to as string) || '';
```

```

const subject = (input.config.subject as string) || 'No Subject';
const bodyTemplate = (input.config.body as string) || '';

if (!to) {
  throw new Error('Email node is missing a recipient address. Please configure
  "To" in the node settings.');
```

```

}

const smtpUser = process.env.SMTP_USER;
const smtpPass = process.env.SMTP_PASS;
const smtpHost = process.env.SMTP_HOST || 'smtp.gmail.com';
const smtpPort = parseInt(process.env.SMTP_PORT || '587', 10);
const smtpFrom = process.env.SMTP_FROM || smtpUser;

if (!smtpUser || !smtpPass) {
  throw new Error(
    'Email node requires SMTP credentials. ' +
    'Add SMTP_USER and SMTP_PASS to packages/server/.env. ' +
    'For Gmail, create an App Password at myaccount.google.com/apppasswords'
  );
}

// Interpolate {input} placeholder in the body
const inputStr = typeof input.previousOutput === 'object'
  ? JSON.stringify(input.previousOutput, null, 2)
  : String(input.previousOutput || '');
const htmlBody = bodyTemplate.replace(/\{input\}/g, inputStr);

logger.info(`📧 [${input.label}] Sending email via ${smtpHost} to "${to}"`);

const transporter = nodemailer.createTransport({
  host: smtpHost,
  port: smtpPort,
  secure: smtpPort === 465,
  auth: {
    user: smtpUser,
    pass: smtpPass,
  },
});

const info = await transporter.sendMail({
  from: `"FlowForge" <${smtpFrom}>`,
  to,
  subject,
  text: htmlBody.replace(/<[^>]+>/g, ''),
  html: `<div style="font-family:sans-serif;max-width:600px;margin:auto">
    ${htmlBody.replace(/\n/g, '<br>')}
    <hr style="margin-top:32px;border-color:#e2e8f0">
    <p style="color:#94a3b8;font-size:12px">Sent by FlowForge
Automation</p>
  </div>`,
});

```

```

logger.info(`📧 [${input.label}] Email sent! Message ID: ${info.messageId}`);

return {
  sent: true,
  to,
  subject,
  messageId: info.messageId,
  accepted: info.accepted,
  timestamp: new Date().toISOString(),
};
}

// ---- DB Query ----
// Executes SQL against a user-provided external Postgres database.
// Requires DB_QUERY_CONNECTION_STRING env var.
async function executeDbQuery(input: NodeExecutionInput): Promise<unknown> {
  const query = (input.config.query as string) || '';

  if (!query) {
    throw new Error('DB Query node is missing a SQL query string.');
```

```

        rows: result.rows,
        fields: result.fields.map((f: { name: string }) => f.name),
        query,
    };
} catch (err) {
    await pool.end();
    throw err;
}
}

// ---- AI LLM (Groq via LangChain) ----
async function executeAiLLM(input: NodeExecutionInput): Promise<unknown> {
    const modelName = (input.config.model as string) || 'llama3-8b-8192';
    const promptTemplate = (input.config.prompt as string) || 'Process this input: {input}';
    const systemPrompt = (input.config.systemPrompt as string) || 'You are a helpful assistant integrated into an automated workflow.';
    const temperature = (input.config.temperature as number) ?? 0.7;

    if (!process.env.GROQ_API_KEY) {
        throw new Error('GROQ_API_KEY environment variable is not set.');
```

```

// ---- Delay ----
// Real timed wait.
async function executeDelay(input: NodeExecutionInput): Promise<unknown> {
  const duration = (input.config.duration as number) || 1;
  const unit = (input.config.unit as string) || 'seconds';
  let ms = duration * 1000;
  if (unit === 'minutes') ms = duration * 60000;
  if (unit === 'ms') ms = duration;
  ms = Math.min(ms, 60000); // Hard cap at 60s to prevent job timeouts

  logger.info(`🕒 [${input.label}] Waiting ${duration} ${unit} (${ms}ms)`);
  await new Promise((resolve) => setTimeout(resolve, ms));

  return {
    delayed: true,
    duration,
    unit,
    waitedMs: ms,
    passedData: input.previousOutput,
  };
}

// ---- Condition ----
// Evaluates a logical JS expression and determines the branch.
async function executeCondition(input: NodeExecutionInput): Promise<unknown> {
  const expression = (input.config.expression as string) || 'true';
  logger.info(`🔍 [${input.label}] Evaluating: ${expression}`);

  let result = true;
  try {
    const func = new Function('input', `return Boolean(${expression})`);
    result = func(input.previousOutput);
  } catch {
    result = true;
  }

  logger.info(`🔍 [${input.label}] → ${result ? 'true' : 'false'} branch`);

  return {
    condition: expression,
    result,
    branch: result ? 'true' : 'false',
    passedData: input.previousOutput,
  };
}

// ---- Data Transform ----
// Runs a JS function expression to transform the incoming data.
async function executeDataTransform(input: NodeExecutionInput): Promise<unknown> {
  const expression = (input.config.expression as string) || 'return input';
  logger.info(`🔄 [${input.label}] Transforming data`);

```

```

    try {
      const func = new Function('input', expression);
      const result = func(input.previousOutput);
      logger.info(`🔧 [${input.label}] Transform succeeded`);
      return result;
    } catch (err) {
      throw new Error(`Data transform error: ${err instanceof Error ? err.message : err}`);
    }
  }

// ---- Code Execute ----
// Runs user-provided JavaScript code with the previous node's output as `input`.
async function executeCodeExecute(input: NodeExecutionInput): Promise<unknown> {
  const code = (input.config.code as string) || 'return { result: "executed" }';
  logger.info(`🔧 [${input.label}] Executing custom code`);

  try {
    const func = new Function('input', code);
    const result = func(input.previousOutput);
    logger.info(`🔧 [${input.label}] Code executed successfully`);
    return result;
  } catch (err) {
    throw new Error(`Code execution failed: ${err instanceof Error ? err.message : err}`);
  }
}

// ---- Slack Notify ----
// POSTs a REAL message to Slack via an Incoming Webhook URL.
// Users configure their webhookUrl in the node's config panel.
// Get a webhook at: api.slack.com/apps → Incoming Webhooks
async function executeSlackNotify(input: NodeExecutionInput): Promise<unknown> {
  const webhookUrl = (input.config.webhookUrl as string) || '';
  const channelDisplay = (input.config.channel as string) || '#general';
  const messageTemplate = (input.config.message as string) || 'Workflow notification from FlowForge';

  if (!webhookUrl) {
    throw new Error(
      'Slack node requires a Webhook URL. ' +
      'Create one at api.slack.com/apps → Your App → Incoming Webhooks, ' +
      'then paste it in the node config panel under "Webhook URL".'
    );
  }

  if (!webhookUrl.startsWith('https://hooks.slack.com/')) {
    throw new Error('Invalid Slack Webhook URL. It must start with https://hooks.slack.com/');
  }
}

```

```

const inputStr = typeof input.previousOutput === 'object'
  ? JSON.stringify(input.previousOutput, null, 2)
  : String(input.previousOutput || '');
const finalMessage = messageTemplate.replace(/\{input\}/g, inputStr);

logger.info(`💬 [${input.label}] Posting to Slack channel: ${channelDisplay}`);

const response = await fetch(webhookUrl, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    text: finalMessage,
    username: 'FlowForge',
    icon_emoji: ':zap:',
  }),
  signal: AbortSignal.timeout(10000),
});

if (!response.ok) {
  const body = await response.text();
  throw new Error(`Slack API error ${response.status}: ${body}`);
}

logger.info(`💬 [${input.label}] Slack message delivered to ${channelDisplay}`);

return {
  sent: true,
  channel: channelDisplay,
  message: finalMessage,
  timestamp: new Date().toISOString(),
};
}

```

Line-by-Line Breakdown

- **Line 8:** `import nodemailer from 'nodemailer';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 9:** `import { ChatGroq } from '@langchain/groq';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 10:** `import { HumanMessage, SystemMessage } from '@langchain/core/messages';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 11:** `import { logger } from '../utils/logger';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 13:** `export interface NodeExecutionInput {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 14:** `nodeId: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 15:** `nodeType: string;`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 16:** `label: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 17:** `config: Record<string, unknown>;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 18:** `previousOutput: unknown;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 19:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 21:** `export interface NodeExecutionResult {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 22:** `success: boolean;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 23:** `output: unknown;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 24:** `error?: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 25:** `duration: number;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 26:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 31:** `export async function executeNode(input: NodeExecutionInput): Promise<NodeExecutionResult> {`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
- **Line 32:** `const startTime = Date.now();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 34:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 35:** `let output: unknown;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 37:** `switch (input.nodeType) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 38:** `case 'webhook-trigger':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 39:** `output = await executeWebhookTrigger(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 40:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 41:** `case 'http-request':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 42:** `output = await executeHttpRequest(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 43:** `break;`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 44:** `case 'email':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 45:** `output = await executeEmail(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 46:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 47:** `case 'db-query':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 48:** `output = await executeDbQuery(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 49:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 50:** `case 'ai-llm':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 51:** `output = await executeAiLlm(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 52:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 53:** `case 'delay':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 54:** `output = await executeDelay(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 55:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 56:** `case 'condition':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 57:** `output = await executeCondition(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 58:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 59:** `case 'data-transform':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 60:** `output = await executeDataTransform(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 61:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 62:** `case 'code-execute':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 63:** `output = await executeCodeExecute(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 64:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 65:** `case 'slack-notify':`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 66:** `output = await executeSlackNotify(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 67:** `break;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 68:** `default:`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 69:** `throw new Error( Unknown node type: ${input.nodeType} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 70:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 72:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 73:** `success: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 74:** `output,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 75:** `duration: Date.now() - startTime,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 76:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 77:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 78:** `const duration = Date.now() - startTime;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 79:** `const errMsg = error instanceof Error ? error.message : 'Unknown error';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 80:** `logger.error( ❌ Node "${input.label}" failed: ${errMsg} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 81:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 82:** `success: false,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 83:** `output: null,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 84:** `error: errMsg,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 85:** `duration,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 86:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 87:** `}`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 88:** }
- *Explanation:* Executes internal logical operations for this block of code.
- **Line 96:** `async function executeWebhookTrigger(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 97:** `logger.info(  [`${input.label}`] Webhook trigger activated );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 98:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 99:** `method: input.config.method || 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 100:** `path: input.config.path || '/webhook',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 101:** `body: input.previousOutput || { message: 'Webhook triggered' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 102:** `headers: { 'content-type': 'application/json' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 103:** `timestamp: new Date().toISOString(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 104:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 105:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 109:** `async function executeHttpRequest(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 110:** `const method = ((input.config.method as string) || 'GET').toUpperCase();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 111:** `const url = (input.config.url as string) || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 112:** `const customHeaders = (input.config.headers as Record<string, string>) || {};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 113:** `const bodyTemplate = input.config.body as string | undefined;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 115:** `if (!url) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 116:** `throw new Error('HTTP Request node is missing a URL. Please configure the URL in the node settings.'...)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 117:** `}`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 119:** `logger.info( 🌐 [{input.label}] ${method} → ${url} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 122:** `let body: string | undefined;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 123:** `if (['POST', 'PUT', 'PATCH'].includes(method) && bodyTemplate) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 124:** `const inputStr = typeof input.previousOutput === 'object'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 125:** `? JSON.stringify(input.previousOutput)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 126:** `: String(input.previousOutput || '');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 127:** `body = bodyTemplate.replace(/\{input\}/g, inputStr);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 128:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 130:** `const response = await fetch(url, {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 131:** `method,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 132:** `headers: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 133:** `'Content-Type': 'application/json',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 134:** `...customHeaders,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 135:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 136:** `...(body ? { body } : {}),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 137:** `signal: AbortSignal.timeout(30000),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 138:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 140:** `let data: unknown;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 141:** `const contentType = response.headers.get('content-type') || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 142:** `if (contentType.includes('application/json')) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 143:** `data = await response.json();`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 144:** `} else {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 145:** `data = await response.text();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 146:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 148:** `logger.info( 🌐 [${input.label}] Response: HTTP ${response.status} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 150:** `if (!response.ok) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 151:** `throw new Error( HTTP ${response.status} ${response.statusText} from ${url} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 152:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 154:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 155:** `status: response.status,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 156:** `statusText: response.statusText,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 157:** `url,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 158:** `method,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 159:** `data,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 160:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 161:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 166:** `async function executeEmail(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 167:** `const to = (input.config.to as string) || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 168:** `const subject = (input.config.subject as string) || 'No Subject';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 169:** `const bodyTemplate = (input.config.body as string) || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 171:** `if (!to) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 172:** `throw new Error('Email node is missing a recipient address. Please configure "To" in the node settin...)`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 173:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 175:** `const smtpUser = process.env.SMTP_USER;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 176:** `const smtpPass = process.env.SMTP_PASS;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 177:** `const smtpHost = process.env.SMTP_HOST || 'smtp.gmail.com';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 178:** `const smtpPort = parseInt(process.env.SMTP_PORT || '587', 10);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 179:** `const smtpFrom = process.env.SMTP_FROM || smtpUser;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 181:** `if (!smtpUser || !smtpPass) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 182:** `throw new Error(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 183:** `'Email node requires SMTP credentials. ' +`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 184:** `'Add SMTP_USER and SMTP_PASS to packages/server/.env. ' +`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 185:** `'For Gmail, create an App Password at myaccount.google.com/apppasswords'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 186:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 187:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 190:** `const inputStr = typeof input.previousOutput === 'object'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 191:** `? JSON.stringify(input.previousOutput, null, 2)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 192:** `: String(input.previousOutput || '');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 193:** `const htmlBody = bodyTemplate.replace(/\{input\}/g, inputStr);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 195:** `logger.info(  [`${input.label}`] Sending email via ${smtpHost} to "${to}" );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 197:** `const transporter = nodemailer.createTransport({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 198:** `host: smtpHost,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 199:** `port: smtpPort,`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 200:** `secure: smtpPort === 465,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 201:** `auth: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 202:** `user: smtpUser,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 203:** `pass: smtpPass,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 204:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 205:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 207:** `const info = await transporter.sendMail({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 208:** `from: "FlowForge" <${smtpFrom}> ,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 209:** `to,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 210:** `subject,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 211:** `text: htmlBody.replace(/<[^>]+>/g, ''),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 212:** `html:`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 213:** `${htmlBody.replace(/\n/g, '<br>')}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 214:** `<hr style="margin-top:32px;border-color:#e2e8f0">`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 215:** `<p style="color:#94a3b8;font-size:12px">Sent by FlowForge Automation</p>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 216:** `</div> ,``
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 217:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 219:** `logger.info( [${input.label}] Email sent! Message ID: ${info.messageId} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 221:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 222:** `sent: true,`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 223:** `to,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 224:** `subject,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 225:** `messageId: info.messageId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 226:** `accepted: info.accepted,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 227:** `timestamp: new Date().toISOString(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 228:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 229:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 234:** `async function executeDbQuery(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 235:** `const query = (input.config.query as string) || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 237:** `if (!query) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 238:** `throw new Error('DB Query node is missing a SQL query string.');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 239:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 241:** `const connectionString = process.env.DB_QUERY_CONNECTION_STRING;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 242:** `if (!connectionString) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 243:** `throw new Error(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 244:** `'DB Query node requires DB_QUERY_CONNECTION_STRING in packages/server/.env. ' +`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 245:** `'Format: postgresql://user:password@host:port/database'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 246:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 247:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 251:** `let Pool: any;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 252:** `try {`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 254:** `Pool = eval("require")('pg').Pool;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 255:** `} catch {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 256:** `throw new Error('The "pg" package is not installed. Run: cd packages/server && npm install pg');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 257:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 258:** `const pool = new Pool({ connectionString });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 259:** `logger.info( [${input.label}] Executing query: ${query.slice(0, 80)}... );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 261:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 262:** `const result = await pool.query(query);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 263:** `await pool.end();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 264:** `logger.info( [${input.label}] Query returned ${result.rowCount} rows );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 265:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 266:** `rowCount: result.rowCount,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 267:** `rows: result.rows,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 268:** `fields: result.fields.map((f: { name: string }) => f.name),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 269:** `query,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 270:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 271:** `} catch (err) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 272:** `await pool.end();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 273:** `throw err;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 274:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 275:** `}`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 278:** `async function executeAiLLm(input: NodeExecutionInput): Promise<unknown>`
`{`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 279:** `const modelName = (input.config.model as string) || 'llama3-8b-8192';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 280:** `const promptTemplate = (input.config.prompt as string) || 'Process this input: {input}';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 281:** `const systemPrompt = (input.config.systemPrompt as string) || 'You are a helpful assistant integrate...'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 282:** `const temperature = (input.config.temperature as number) ?? 0.7;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 284:** `if (!process.env.GROQ_API_KEY) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 285:** `throw new Error('GROQ_API_KEY environment variable is not set.');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 286:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 288:** `logger.info(👉 [`${input.label}`] Calling Groq model: ${modelName} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 290:** `const chatModel = new ChatGroq({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 291:** `apiKey: process.env.GROQ_API_KEY,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 292:** `model: modelName,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 293:** `temperature,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 294:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 296:** `const inputDataStr = typeof input.previousOutput === 'object'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 297:** `? JSON.stringify(input.previousOutput, null, 2)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 298:** `: String(input.previousOutput || '');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 299:** `const finalPrompt = promptTemplate.replace(/\{input\}/g, inputDataStr);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 301:** `const messages = [`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 302:** `new SystemMessage(systemPrompt),`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 303:** `new HumanMessage(finalPrompt),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 304:** `];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 306:** `const response = await chatModel.invoke(messages);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 307:** `logger.info( 🗣️ [`${input.label}`] Groq response received );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 309:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 310:** `model: modelName,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 311:** `provider: 'groq',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 312:** `prompt: finalPrompt,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 313:** `response: response.content,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 314:** `usage: response.response_metadata?.tokenUsage || { totalTokens: 0 },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 315:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 316:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 320:** `async function executeDelay(input: NodeExecutionInput): Promise<unknown>`
`{`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 321:** `const duration = (input.config.duration as number) || 1;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 322:** `const unit = (input.config.unit as string) || 'seconds';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 323:** `let ms = duration * 1000;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 324:** `if (unit === 'minutes') ms = duration * 60000;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 325:** `if (unit === 'ms') ms = duration;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 326:** `ms = Math.min(ms, 60000); // Hard cap at 60s to prevent job timeouts`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 328:** `logger.info( ⌚ [`${input.label}`] Waiting ${duration} ${unit} (${ms}ms) );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 329:** `await new Promise((resolve) => setTimeout(resolve, ms));`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 331:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 332:** `delayed: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 333:** `duration,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 334:** `unit,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 335:** `waitedMs: ms,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 336:** `passedData: input.previousOutput,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 337:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 338:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 342:** `async function executeCondition(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 343:** `const expression = (input.config.expression as string) || 'true';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 344:** `logger.info(  [{input.label}] Evaluating: ${expression} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 346:** `let result = true;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 347:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 348:** `const func = new Function('input', return Boolean(${expression}) );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 349:** `result = func(input.previousOutput);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 350:** `} catch {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 351:** `result = true;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 352:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 354:** `logger.info(  [{input.label}] → ${result ? 'true' : 'false'} branch );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 356:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 357:** `condition: expression,`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 358:** `result,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 359:** `branch: result ? 'true' : 'false',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 360:** `passedData: input.previousOutput,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 361:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 362:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 366:** `async function executeDataTransform(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 367:** `const expression = (input.config.expression as string) || 'return input';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 368:** `logger.info( ⓘ [`${input.label}`] Transforming data );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 370:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 371:** `const func = new Function('input', expression);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 372:** `const result = func(input.previousOutput);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 373:** `logger.info( ⓘ [`${input.label}`] Transform succeeded );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 374:** `return result;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 375:** `} catch (err) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 376:** `throw new Error( Data transform error: ${err instanceof Error ? err.message : err} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 377:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 378:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 382:** `async function executeCodeExecute(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 383:** `const code = (input.config.code as string) || 'return { result: "executed" }';`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 384:** `logger.info( █ [${input.label}] Executing custom code );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 386:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 387:** `const func = new Function('input', code);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 388:** `const result = func(input.previousOutput);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 389:** `logger.info( █ [${input.label}] Code executed successfully );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 390:** `return result;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 391:** `} catch (err) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 392:** `throw new Error( Code execution failed: ${err instanceof Error ? err.message : err} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 393:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 394:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 400:** `async function executeSlackNotify(input: NodeExecutionInput): Promise<unknown> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 401:** `const webhookUrl = (input.config.webhookUrl as string) || '';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 402:** `const channelDisplay = (input.config.channel as string) || '#general';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 403:** `const messageTemplate = (input.config.message as string) || 'Workflow notification from FlowForge';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 405:** `if (!webhookUrl) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 406:** `throw new Error(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 407:** `'Slack node requires a Webhook URL. ' +`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 408:** `'Create one at api.slack.com/apps → Your App → Incoming Webhooks, ' +`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 409:** `'then paste it in the node config panel under "Webhook URL".'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 410:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 411:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 413:** `if (!webhookUrl.startsWith('https://hooks.slack.com/')) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 414:** `throw new Error('Invalid Slack Webhook URL. It must start with https://hooks.slack.com/');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 415:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 417:** `const inputStr = typeof input.previousOutput === 'object'`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 418:** `? JSON.stringify(input.previousOutput, null, 2)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 419:** `: String(input.previousOutput || '');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 420:** `const finalMessage = messageTemplate.replace(/\{input\}/g, inputStr);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 422:** `logger.info(💬 [`${input.label}`] Posting to Slack channel: ${channelDisplay} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 424:** `const response = await fetch(webhookUrl, {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 425:** `method: 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 426:** `headers: { 'Content-Type': 'application/json' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 427:** `body: JSON.stringify({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 428:** `text: finalMessage,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 429:** `username: 'FlowForge',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 430:** `icon_emoji: ':zap:',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 431:** `}),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 432:** `signal: AbortSignal.timeout(10000),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 433:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 435:** `if (!response.ok) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 436:** `const body = await response.text();`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 437:** `throw new Error( Slack API error ${response.status}: ${body} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 438:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 440:** `logger.info( [${input.label}] Slack message delivered to ${channelDisplay} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 442:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 443:** `sent: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 444:** `channel: channelDisplay,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 445:** `message: finalMessage,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 446:** `timestamp: new Date().toISOString(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 447:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 448:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.

File: `packages/server/src/services/workflowExecutor.ts`

Source Code

```
// =====
// FlowForge – Workflow Executor (BullMQ Worker)
// =====
// This is the engine that runs workflows. It:
// 1. Receives a workflow ID from the BullMQ queue
// 2. Loads the workflow (nodes + edges) from the database
// 3. Topologically sorts the nodes (respecting DAG order)
// 4. Executes each node in sequence, passing output to the next
// 5. Tracks status and emits real-time updates via Socket.IO
//
// WHAT IS TOPOLOGICAL SORT?
// In a DAG (Directed Acyclic Graph), nodes must run in dependency order.
// If Node A → Node B → Node C, then A runs first, then B, then C.
// Topological sort figures out this order automatically.

import { Queue, Worker, Job } from 'bullmq';
import { prisma } from '../lib/prisma';
import { executeNode, NodeExecutionInput } from '../services/nodeExecutor';
import { emitExecutionStatus, emitNodeStatus, emitExecutionLog } from
'../lib/socketio';
import { logger } from '../utils/logger';
```

```

const REDIS_URL = process.env.REDIS_URL || 'redis://localhost:6379';

// Parse Redis URL into connection options
function getRedisConnection() {
  const url = new URL(REDIS_URL);
  return {
    host: url.hostname,
    port: parseInt(url.port) || 6379,
  };
}

// ---- BullMQ Queue ----
// Jobs are added to this queue when a user clicks "Run"
export const workflowQueue = new Queue('workflow-execution', {
  connection: getRedisConnection(),
  defaultJobOptions: {
    removeOnComplete: 100, // Keep last 100 completed jobs
    removeOnFail: 50,
    attempts: 1,
  },
});

// ---- Interfaces ----
interface FlowNode {
  id: string;
  type: string;
  data: {
    nodeType: string;
    label: string;
    config: Record<string, unknown>;
  };
  position: { x: number; y: number };
}

interface FlowEdge {
  id: string;
  source: string;
  target: string;
  sourceHandle?: string | null;
}

// ---- Topological Sort ----
// Determines the execution order of nodes based on edges
function topologicalSort(nodes: FlowNode[], edges: FlowEdge[]): FlowNode[] {
  const nodeMap = new Map(nodes.map((n) => [n.id, n]));
  const inDegree = new Map<string, number>();
  const adjacency = new Map<string, string[]>();

  // Initialize
  for (const node of nodes) {
    inDegree.set(node.id, 0);
  }

```

```

    adjacency.set(node.id, []);
}

// Count incoming edges for each node
for (const edge of edges) {
    const count = inDegree.get(edge.target) || 0;
    inDegree.set(edge.target, count + 1);
    const adj = adjacency.get(edge.source) || [];
    adj.push(edge.target);
    adjacency.set(edge.source, adj);
}

// Start with nodes that have no incoming edges (in-degree 0)
const queue: string[] = [];
for (const [nodeId, degree] of inDegree) {
    if (degree === 0) queue.push(nodeId);
}

const sorted: FlowNode[] = [];
while (queue.length > 0) {
    const nodeId = queue.shift()!;
    const node = nodeMap.get(nodeId);
    if (node) sorted.push(node);

    const neighbors = adjacency.get(nodeId) || [];
    for (const neighbor of neighbors) {
        const newDegree = (inDegree.get(neighbor) || 1) - 1;
        inDegree.set(neighbor, newDegree);
        if (newDegree === 0) queue.push(neighbor);
    }
}

return sorted;
}

// ---- BullMQ Worker ----
// Processes workflow execution jobs
export function startWorkflowWorker() {
    const worker = new Worker(
        'workflow-execution',
        async (job: Job) => {
            const { workflowId, executionId } = job.data;

            logger.info(`🚀 Starting execution ${executionId} for workflow ${workflowId}`);
            emitExecutionStatus(executionId, 'RUNNING');
            emitExecutionLog(executionId, 'info', `🚀 Workflow execution started`);

            try {
                // 1. Load the workflow from the database
                const workflow = await prisma.workflow.findUnique({
                    where: { id: workflowId },

```

```

});

if (!workflow) {
  throw new Error(`Workflow ${workflowId} not found`);
}

const nodes = (workflow.nodesJson as unknown as FlowNode[]) || [];
const edges = (workflow.edgesJson as unknown as FlowEdge[]) || [];

if (nodes.length === 0) {
  throw new Error('Workflow has no nodes to execute');
}

// 2. Topologically sort the nodes
const sortedNodes = topologicalSort(nodes, edges);
emitExecutionLog(
  executionId,
  'info',
  `📋 Execution plan: ${sortedNodes.map((n) => n.data.label).join(' → ')}`,
);

// 3. Execute each node in order
const nodeOutputs = new Map<string, unknown>();
let hasFailure = false;

// Update execution with total node count
await prisma.execution.update({
  where: { id: executionId },
  data: { status: 'RUNNING' },
});

for (const node of sortedNodes) {
  // Mark node as RUNNING
  emitNodeStatus(executionId, node.id, 'RUNNING');
  emitExecutionLog(executionId, 'info', `▶ Running: ${node.data.label}`,
node.id);

  // Create node execution record
  const nodeExecution = await prisma.nodeExecution.create({
    data: {
      executionId,
      nodeId: node.id,
      nodeType: node.data.nodeType,
      nodeName: node.data.label,
      status: 'RUNNING',
      startedAt: new Date(),
    },
  });

  // Find the previous node's output (follow edges)
  const incomingEdge = edges.find((e) => e.target === node.id);
  const previousOutput = incomingEdge

```

```

    ? nodeOutputs.get(incomingEdge.source)
    : null;

// Execute the node
const input: NodeExecutionInput = {
  nodeId: node.id,
  nodeType: node.data.nodeType,
  label: node.data.label,
  config: node.data.config || {},
  previousOutput,
};

const result = await executeNode(input);

// Store the output
nodeOutputs.set(node.id, result.output);

// Update node execution record
await prisma.nodeExecution.update({
  where: { id: nodeExecution.id },
  data: {
    status: result.success ? 'SUCCESS' : 'FAILED',
    outputData: result.output as object || undefined,
    errorMessage: result.error || undefined,
    duration: result.duration,
    completedAt: new Date(),
  },
});

if (result.success) {
  emitNodeStatus(executionId, node.id, 'COMPLETED', {
    duration: result.duration,
  });
  emitExecutionLog(
    executionId,
    'success',
    `✅ Completed: ${node.data.label} (${result.duration}ms)`,
    node.id,
  );
} else {
  emitNodeStatus(executionId, node.id, 'FAILED', {
    error: result.error,
  });
  emitExecutionLog(
    executionId,
    'error',
    `❌ Failed: ${node.data.label} - ${result.error}`,
    node.id,
  );
  hasFailure = true;
  break; // Stop execution on failure
}

```

```

    }

    // 4. Mark execution as completed or failed
    const finalStatus = hasFailure ? 'FAILED' : 'COMPLETED';
    await prisma.execution.update({
      where: { id: executionId },
      data: {
        status: finalStatus,
        completedAt: new Date(),
      },
    });

    emitExecutionStatus(executionId, finalStatus);
    emitExecutionLog(
      executionId,
      hasFailure ? 'error' : 'success',
      hasFailure ? '❌ Workflow execution failed' : '🎉 Workflow execution
completed!',
    );

    logger.success(`✅ Execution ${executionId} ${finalStatus}`);
  } catch (error) {
    const errorMsg = error instanceof Error ? error.message : 'Unknown error';
    logger.error(`❌ Execution ${executionId} failed: ${errorMsg}`);

    await prisma.execution.update({
      where: { id: executionId },
      data: {
        status: 'FAILED',
        completedAt: new Date(),
      },
    });

    emitExecutionStatus(executionId, 'FAILED', { error: errorMsg });
    emitExecutionLog(executionId, 'error', `❌ Fatal error: ${errorMsg}`);
  }
},
{
  connection: getRedisConnection(),
  concurrency: 5, // Run up to 5 workflows simultaneously
},
);

worker.on('completed', (job) => {
  logger.info(`📦 Job ${job.id} completed`);
});

worker.on('failed', (job, error) => {
  logger.error(`📦 Job ${job?.id} failed: ${error.message}`);
});

logger.success(`🚀 Workflow execution worker started`);

```

```
    return worker;
}
```

Line-by-Line Breakdown

- **Line 16:** `import { Queue, Worker, Job } from 'bullmq';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 17:** `import { prisma } from '../lib/prisma';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 18:** `import { executeNode, NodeExecutionInput } from '../services/nodeExecutor';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 19:** `import { emitExecutionStatus, emitNodeStatus, emitExecutionLog } from '../lib/socketio';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 20:** `import { logger } from '../utils/logger';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 22:** `const REDIS_URL = process.env.REDIS_URL || 'redis://localhost:6379';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 25:** `function getRedisConnection() {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 26:** `const url = new URL(REDIS_URL);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 27:** `return {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 28:** `host: url.hostname,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 29:** `port: parseInt(url.port) || 6379,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 30:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 31:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 35:** `export const workflowQueue = new Queue('workflow-execution', {`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
- **Line 36:** `connection: getRedisConnection(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 37:** `defaultJobOptions: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 38:** `removeOnComplete: 100, // Keep last 100 completed jobs`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 39:** `removeOnFail: 50,`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 40:** `attempts: 1,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 41:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 42:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 45:** `interface FlowNode {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 46:** `id: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 47:** `type: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 48:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 49:** `nodeType: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 50:** `label: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 51:** `config: Record<string, unknown>;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 52:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 53:** `position: { x: number; y: number };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 54:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 56:** `interface FlowEdge {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 57:** `id: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 58:** `source: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 59:** `target: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 60:** `sourceHandle?: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 61:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 65:** `function topologicalSort(nodes: FlowNode[], edges: FlowEdge[]): FlowNode[] {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 66:** `const nodeMap = new Map(nodes.map((n) => [n.id, n]));`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 67:** `const inDegree = new Map<string, number>();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 68:** `const adjacency = new Map<string, string[]>();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 71:** `for (const node of nodes) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 72:** `inDegree.set(node.id, 0);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 73:** `adjacency.set(node.id, []);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 74:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 77:** `for (const edge of edges) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 78:** `const count = inDegree.get(edge.target) || 0;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 79:** `inDegree.set(edge.target, count + 1);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 80:** `const adj = adjacency.get(edge.source) || [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 81:** `adj.push(edge.target);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 82:** `adjacency.set(edge.source, adj);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 83:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 86:** `const queue: string[] = [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 87:** `for (const [nodeId, degree] of inDegree) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 88:** `if (degree === 0) queue.push(nodeId);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 89:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 91:** `const sorted: FlowNode[] = [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 92:** `while (queue.length > 0) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 93:** `const nodeId = queue.shift()!;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 94:** `const node = nodeMap.get(nodeId);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 95:** `if (node) sorted.push(node);`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 97:** `const neighbors = adjacency.get(nodeId) || [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 98:** `for (const neighbor of neighbors) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 99:** `const newDegree = (inDegree.get(neighbor) || 1) - 1;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 100:** `inDegree.set(neighbor, newDegree);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 101:** `if (newDegree === 0) queue.push(neighbor);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 102:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 103:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 105:** `return sorted;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 106:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 110:** `export function startWorkflowWorker() {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 111:** `const worker = new Worker(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 112:** `'workflow-execution',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 113:** `async (job: Job) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 114:** `const { workflowId, executionId } = job.data;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 116:** `logger.info( 🚀 Starting execution ${executionId} for workflow ${workflowId} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 117:** `emitExecutionStatus(executionId, 'RUNNING');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 118:** `emitExecutionLog(executionId, 'info', ' 🚀 Workflow execution started');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 120:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 122:** `const workflow = await prisma.workflow.findUnique({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 123:** `where: { id: workflowId },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 124:** `});`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 126:** `if (!workflow) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 127:** `throw new Error( Workflow ${workflowId} not found );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 128:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 130:** `const nodes = (workflow.nodesJson as unknown as FlowNode[]) || [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 131:** `const edges = (workflow.edgesJson as unknown as FlowEdge[]) || [];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 133:** `if (nodes.length === 0) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 134:** `throw new Error('Workflow has no nodes to execute');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 135:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 138:** `const sortedNodes = topologicalSort(nodes, edges);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 139:** `emitExecutionLog(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 140:** `executionId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 141:** `'info',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 142:** ``📄 Execution plan: ${sortedNodes.map((n) => n.data.label).join(' → ')} ,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 143:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 146:** `const nodeOutputs = new Map<string, unknown>();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 147:** `let hasFailure = false;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 150:** `await prisma.execution.update({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 151:** `where: { id: executionId },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 152:** `data: { status: 'RUNNING' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 153:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 155:** `for (const node of sortedNodes) {`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 157:** `emitNodeStatus(executionId, node.id, 'RUNNING');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 158:** `emitExecutionLog(executionId, 'info',  Running: ${node.data.label} , node.id);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 161:** `const nodeExecution = await prisma.nodeExecution.create({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 162:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 163:** `executionId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 164:** `nodeId: node.id,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 165:** `nodeType: node.data.nodeType,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 166:** `nodeName: node.data.label,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 167:** `status: 'RUNNING',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 168:** `startedAt: new Date(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 169:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 170:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 173:** `const incomingEdge = edges.find((e) => e.target === node.id);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 174:** `const previousOutput = incomingEdge`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 175:** `? nodeOutputs.get(incomingEdge.source)`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 176:** `: null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 179:** `const input: NodeExecutionInput = {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 180:** `nodeId: node.id,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 181:** `nodeType: node.data.nodeType,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 182:** `label: node.data.label,`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 183:** `config: node.data.config || {},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 184:** `previousOutput,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 185:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 187:** `const result = await executeNode(input);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 190:** `nodeOutputs.set(node.id, result.output);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 193:** `await prisma.nodeExecution.update({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 194:** `where: { id: nodeExecution.id },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 195:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 196:** `status: result.success ? 'SUCCESS' : 'FAILED',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 197:** `outputData: result.output as object || undefined,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 198:** `errorMessage: result.error || undefined,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 199:** `duration: result.duration,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 200:** `completedAt: new Date(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 201:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 202:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 204:** `if (result.success) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 205:** `emitNodeStatus(executionId, node.id, 'COMPLETED', {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 206:** `duration: result.duration,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 207:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 208:** `emitExecutionLog(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 209:** `executionId,`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 210:** `'success',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 211:** ```✅ Completed: ${node.data.label} (${result.duration}ms) ,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 212:** `node.id,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 213:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 214:** `} else {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 215:** `emitNodeStatus(executionId, node.id, 'FAILED', {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 216:** `error: result.error,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 217:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 218:** `emitExecutionLog(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 219:** `executionId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 220:** `'error',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 221:** ```❌ Failed: ${node.data.label} — ${result.error} ,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 222:** `node.id,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 223:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 224:** `hasFailure = true;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 225:** `break; // Stop execution on failure`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 226:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 227:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 230:** `const finalStatus = hasFailure ? 'FAILED' : 'COMPLETED';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 231:** `await prisma.execution.update({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 232:** `where: { id: executionId },`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 233:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 234:** `status: finalStatus,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 235:** `completedAt: new Date(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 236:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 237:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 239:** `emitExecutionStatus(executionId, finalStatus);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 240:** `emitExecutionLog(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 241:** `executionId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 242:** `hasFailure ? 'error' : 'success',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 243:** `hasFailure ? '❌ Workflow execution failed' : '🎉 Workflow execution completed!',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 244:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 246:** `logger.success(✅ Execution ${executionId} ${finalStatus} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 247:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 248:** `const errorMsg = error instanceof Error ? error.message : 'Unknown error';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 249:** `logger.error(❌ Execution ${executionId} failed: ${errorMsg} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 251:** `await prisma.execution.update({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 252:** `where: { id: executionId },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 253:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 254:** `status: 'FAILED',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 255:** `completedAt: new Date(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 256:** `},`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 257:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 259:** `emitExecutionStatus(executionId, 'FAILED', { error: errorMsg });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 260:** `emitExecutionLog(executionId, 'error', ✖ Fatal error: ${errorMsg} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 261:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 262:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 263:** `{`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 264:** `connection: getRedisConnection(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 265:** `concurrency: 5, // Run up to 5 workflows simultaneously`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 266:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 267:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 269:** `worker.on('completed', (job) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 270:** `logger.info(  Job ${job.id} completed );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 271:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 273:** `worker.on('failed', (job, error) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 274:** `logger.error(  Job ${job?.id} failed: ${error.message} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 275:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 277:** `logger.success(  Workflow execution worker started');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 278:** `return worker;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 279:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.

File: `packages/server/src/routes/workflows.ts`

Source Code

```
// =====
// FlowForge – Workflow Routes
// =====
// CRUD endpoints for workflows (Create, Read, Update, Delete).
// All routes require authentication (authMiddleware).
//
// GET    /api/workflows      – List all workflows in a workspace
// POST   /api/workflows      – Create a new workflow
// GET    /api/workflows/:id   – Get a single workflow
// PUT    /api/workflows/:id   – Update a workflow (name, nodes, edges, etc.)
import { Router, Response, NextFunction } from 'express';
import { z } from 'zod';
import { prisma } from '../lib/prisma';
import { authMiddleware, AuthRequest } from '../middleware/auth';
import { AppError } from '../middleware/errorHandler';
import { validateRequest } from '../middleware/validate';
import {
  createWorkflowSchema,
  updateWorkflowSchema,
  getWorkflowSchema,
  deleteWorkflowSchema,
} from '../schemas/workflow';

const router = Router();

// All workflow routes require authentication
router.use(authMiddleware);

// ---- GET /api/workflows ----
// Lists all workflows for a given workspace
router.get('/', async (req: AuthRequest, res: Response, next: NextFunction) => {
  try {
    const workspaceId = req.query.workspaceId as string;
    if (!workspaceId) {
      throw new AppError('workspaceId query parameter is required', 400);
    }

    const workflows = await prisma.workflow.findMany({
      where: { workspaceId },
      orderBy: { updatedAt: 'desc' },
      select: {
        id: true,
        name: true,
        description: true,
        status: true,
        triggerType: true,
        createdAt: true,
        updatedAt: true,
        createdBy: {
          select: { id: true, name: true, avatarUrl: true },
        },
      },
    });
  }
}
```

```

        _count: {
          select: { executions: true },
        },
      },
    });

    res.json({ success: true, data: workflows });
  } catch (error) {
    next(error);
  }
});

// Validation schemas imported from ../schemas/workflow.ts

// ---- POST /api/workflows ----
// Creates a new workflow
router.post(
  '/',
  validateRequest(createWorkflowSchema),
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    try {
      const { name, description, workspaceId } = req.body;

      const workflow = await prisma.workflow.create({
        data: {
          name,
          description,
          workspaceId,
          triggerType: 'MANUAL',
          createdById: req.userId!,
          nodesJson: [],
          edgesJson: [],
        },
      });
    } catch (error) {
      next(error);
    }
  }
);

// ---- GET /api/workflows/:id ----
// Gets a single workflow with all its data (including nodes and edges)
router.get(
  '/:id',
  validateRequest(getWorkflowSchema),
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    try {
      const workflow = await prisma.workflow.findUnique({
        where: { id: req.params.id as string },
        include: {

```

```

        createdBy: {
          select: { id: true, name: true, avatarUrl: true },
        },
      },
    });

    if (!workflow) {
      throw new AppError('Workflow not found', 404);
    }

    res.json({ success: true, data: workflow });
  } catch (error) {
    next(error);
  }
});

// ---- PUT /api/workflows/:id ----
// Updates a workflow (used when saving the canvas)
router.put(
 ('/:id',
  validateRequest(updateWorkflowSchema),
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    try {
      const existing = await prisma.workflow.findUnique({
        where: { id: req.params.id as string },
      });

      if (!existing) {
        throw new AppError('Workflow not found', 404);
      }

      const workflow = await prisma.workflow.update({
        where: { id: req.params.id as string },
        data: {
          name: req.body.name ?? existing.name,
          description: req.body.description ?? existing.description,
          status: req.body.status ?? existing.status,
          triggerType: req.body.triggerType ?? existing.triggerType,
          cronExpression: req.body.cronExpression ?? existing.cronExpression,
          nodesJson: req.body.nodesJson ?? existing.nodesJson,
          edgesJson: req.body.edgesJson ?? existing.edgesJson,
        },
      });

      res.json({ success: true, data: workflow });
    } catch (error) {
      next(error);
    }
  });

// ---- DELETE /api/workflows/:id ----
// Deletes a workflow and all related data (cascading)

```

```

router.delete(
 ('/:id',
  validateRequest(deleteWorkflowSchema),
  async (req: AuthRequest, res: Response, next: NextFunction) => {
    try {
      const existing = await prisma.workflow.findUnique({
        where: { id: req.params.id as string },
      });

      if (!existing) {
        throw new AppError('Workflow not found', 404);
      }

      await prisma.workflow.delete({
        where: { id: req.params.id as string },
      });

      res.json({ success: true, message: 'Workflow deleted successfully' });
    } catch (error) {
      next(error);
    }
  });

export default router;

```

Line-by-Line Breakdown

- **Line 11:** `import { Router, Response, NextFunction } from 'express';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 12:** `import { z } from 'zod';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 13:** `import { prisma } from '../lib/prisma';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 14:** `import { authMiddleware, AuthRequest } from '../middleware/auth';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 15:** `import { AppError } from '../middleware/errorHandler';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 16:** `import { validateRequest } from '../middleware/validate';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 17:** `import {`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 18:** `createWorkflowSchema,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 19:** `updateWorkflowSchema,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 20:** `getWorkflowSchema,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 21:** `deleteWorkflowSchema,`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 22:** `} from '../schemas/workflow';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 24:** `const router = Router();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 27:** `router.use(authMiddleware);`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 31:** `router.get('/', async (req: AuthRequest, res: Response, next: NextFunction) => {`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 32:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 33:** `const workspaceId = req.query.workspaceId as string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 34:** `if (!workspaceId) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 35:** `throw new AppError('workspaceId query parameter is required', 400);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 36:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 38:** `const workflows = await prisma.workflow.findMany({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 39:** `where: { workspaceId },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 40:** `orderBy: { updatedAt: 'desc' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 41:** `select: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 42:** `id: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 43:** `name: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 44:** `description: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 45:** `status: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 46:** `triggerType: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 47:** `createdAt: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 48:** `updatedAt: true,`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 49:** `createdBy: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 50:** `select: { id: true, name: true, avatarUrl: true },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 51:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 52:** `_count: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 53:** `select: { executions: true },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 54:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 55:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 56:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 58:** `res.json({ success: true, data: workflows });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 59:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 60:** `next(error);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 61:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 62:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 68:** `router.post(`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 69:** `'/',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 70:** `validateRequest(createWorkflowSchema),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 71:** `async (req: AuthRequest, res: Response, next: NextFunction) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 72:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 73:** `const { name, description, workspaceId } = req.body;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 75:** `const workflow = await prisma.workflow.create({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).

- **Line 76:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 77:** `name,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 78:** `description,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 79:** `workspaceId,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 80:** `triggerType: 'MANUAL',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 81:** `createdById: req.userId!,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 82:** `nodesJson: [],`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 83:** `edgesJson: [],`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 84:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 85:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 87:** `res.status(201).json({ success: true, data: workflow });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 88:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 89:** `next(error);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 90:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 91:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 92:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 96:** `router.get(`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 97:** `('/:id',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 98:** `validateRequest(getWorkflowSchema),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 99:** `async (req: AuthRequest, res: Response, next: NextFunction) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 100:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 101:** `const workflow = await prisma.workflow.findUnique({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 102:** `where: { id: req.params.id as string },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 103:** `include: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 104:** `createdBy: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 105:** `select: { id: true, name: true, avatarUrl: true },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 106:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 107:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 108:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 110:** `if (!workflow) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 111:** `throw new AppError('Workflow not found', 404);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 112:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 114:** `res.json({ success: true, data: workflow });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 115:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 116:** `next(error);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 117:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 118:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 122:** `router.put(`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 123:** `('/:id',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 124:** `validateRequest(updateWorkflowSchema),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 125:** `async (req: AuthRequest, res: Response, next: NextFunction) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 126:** `try {`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 127:** `const existing = await prisma.workflow.findUnique({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 128:** `where: { id: req.params.id as string },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 129:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 131:** `if (!existing) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 132:** `throw new AppError('Workflow not found', 404);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 133:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 135:** `const workflow = await prisma.workflow.update({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 136:** `where: { id: req.params.id as string },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 137:** `data: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 138:** `name: req.body.name ?? existing.name,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 139:** `description: req.body.description ?? existing.description,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 140:** `status: req.body.status ?? existing.status,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 141:** `triggerType: req.body.triggerType ?? existing.triggerType,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 142:** `cronExpression: req.body.cronExpression ?? existing.cronExpression,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 143:** `nodesJson: req.body.nodesJson ?? existing.nodesJson,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 144:** `edgesJson: req.body.edgesJson ?? existing.edgesJson,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 145:** `},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 146:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 148:** `res.json({ success: true, data: workflow });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 149:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 150:** `next(error);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 151:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 152:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 156:** `router.delete(`
 - *Explanation:* Creates an Express.js API route that the frontend UI can send requests to across the internet.
- **Line 157:** `('/:id',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 158:** `validateRequest(deleteWorkflowSchema),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 159:** `async (req: AuthRequest, res: Response, next: NextFunction) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 160:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 161:** `const existing = await prisma.workflow.findUnique({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 162:** `where: { id: req.params.id as string },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 163:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 165:** `if (!existing) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 166:** `throw new AppError('Workflow not found', 404);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 167:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 169:** `await prisma.workflow.delete({`
 - *Explanation:* Triggers the Prisma ORM to safely query the PostgreSQL database (e.g., storing data, searching, or deleting).
- **Line 170:** `where: { id: req.params.id as string },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 171:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 173:** `res.json({ success: true, message: 'Workflow deleted successfully' });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 174:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 175:** `next(error);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 176:** `}`

- *Explanation:* Executes internal logical operations for this block of code.
 - **Line 177:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
 - **Line 179:** `export default router;`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
-

File: `packages/server/src/index.ts`

Source Code

```
// =====  
// FlowForge – Express Server Entry Point  
// =====  
// This is the main file that starts the backend server.  
//  
// WHAT HAPPENS WHEN THE SERVER STARTS:  
// 1. Loads environment variables from .env  
// 2. Creates an Express app with middleware (CORS, JSON parsing, etc.)  
// 3. Sets up Socket.IO for real-time communication  
// 4. Registers all API routes  
// 5. Starts the BullMQ worker for workflow execution  
// 6. Starts listening on the configured port (default: 4000)  
  
import dotenv from 'dotenv';  
import path from 'path';  
  
// Load .env from the server directory first, then fall back to root  
dotenv.config({ path: path.resolve(__dirname, '../.env') });  
dotenv.config({ path: path.resolve(__dirname, '../../.env') });  
  
import express from 'express';  
import cors from 'cors';  
import helmet from 'helmet';  
import { createServer } from 'http';  
import { logger } from './utils/logger';  
import { errorHandler } from './middleware/errorHandler';  
import { initSocketIO } from './lib/socketio';  
import { startWorkflowWorker } from './services/workflowExecutor';  
import authRoutes from './routes/auth';  
import workflowRoutes from './routes/workflows';  
import executionRoutes from './routes/executions';  
  
// ---- Create the Express app ----  
const app = express();  
const httpServer = createServer(app);  
  
// ---- Socket.IO Setup ----  
// Uses our Socket.IO manager (socketio.ts) which provides emit helpers
```

```

// that the executor uses to send real-time updates
const io = initSocketIO(httpServer);

// ---- Middleware ----

// Helmet: Adds security headers to protect against common attacks
app.use(helmet());

// CORS: Allows the frontend (localhost:3000) to call the backend (localhost:4000)
app.use(
  cors({
    origin: process.env.NEXTAUTH_URL || 'http://localhost:3000',
    credentials: true,
  }),
);

// JSON parser: Converts incoming JSON request bodies to JavaScript objects
app.use(express.json({ limit: '10mb' }));

// URL-encoded parser: Handles form submissions
app.use(express.urlencoded({ extended: true }));

// ---- API Routes ----

// Health check – a simple endpoint to verify the server is running
app.get('/api/health', (_req, res) => {
  res.json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
  });
});

// Auth routes: /api/auth/register, /api/auth/login
app.use('/api/auth', authRoutes);

// Workflow routes: /api/workflows (CRUD)
app.use('/api/workflows', workflowRoutes);

// Execution routes: /api/workflows/:id/execute, /api/executions
app.use('/api', executionRoutes);

// Make io accessible to routes (for emitting events)
app.set('io', io);

// ---- Error Handler (must be LAST) ----
app.use(errorHandler);

// ---- Start the Server ----
const PORT = parseInt(process.env.PORT || '4000', 10);

httpServer.listen(PORT, () => {

```

```

logger.success(`🚀 FlowForge server running on http://localhost:${PORT}`);
logger.info(`🗄 Database: PostgreSQL via Prisma`);
logger.info(`🗄 Redis: ${process.env.REDIS_URL || 'redis://localhost:6379'}`);

// Start the BullMQ worker for workflow execution
startWorkflowWorker();
});

// Export for testing
export { app, io, httpServer };

```

Line-by-Line Breakdown

- **Line 14:** `import dotenv from 'dotenv';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 15:** `import path from 'path';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 18:** `dotenv.config({ path: path.resolve(__dirname, '../.env') });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 19:** `dotenv.config({ path: path.resolve(__dirname, '../../.env') });`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 21:** `import express from 'express';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 22:** `import cors from 'cors';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 23:** `import helmet from 'helmet';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 24:** `import { createServer } from 'http';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 25:** `import { logger } from './utils/logger';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 26:** `import { errorHandler } from './middleware/errorHandler';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 27:** `import { initSocketIO } from './lib/socketio';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 28:** `import { startWorkflowWorker } from './services/workflowExecutor';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 29:** `import authRoutes from './routes/auth';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 30:** `import workflowRoutes from './routes/workflows';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 31:** `import executionRoutes from './routes/executions';`
 - *Explanation:* Imports necessary generic dependencies or custom functions from other files.
- **Line 34:** `const app = express();`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 35:** `const httpServer = createServer(app);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 40:** `const io = initSocketIO(httpServer);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 45:** `app.use(helmet());`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 48:** `app.use(`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 49:** `cors({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 50:** `origin: process.env.NEXTAUTH_URL || 'http://localhost:3000',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 51:** `credentials: true,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 52:** `}},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 53:** `);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 56:** `app.use(express.json({ limit: '10mb' }));`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 59:** `app.use(express.urlencoded({ extended: true }));`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 64:** `app.get('/api/health', (_req, res) => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 65:** `res.json({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 66:** `status: 'ok',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 67:** `timestamp: new Date().toISOString(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 68:** `uptime: process.uptime(),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 69:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 70:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 73:** `app.use('/api/auth', authRoutes);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 76:** `app.use('/api/workflows', workflowRoutes);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 79:** `app.use('/api', executionRoutes);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 82:** `app.set('io', io);`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 85:** `app.use(errorHandler);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 88:** `const PORT = parseInt(process.env.PORT || '4000', 10);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 90:** `httpServer.listen(PORT, () => {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 91:** `logger.success(🚀 FlowForge server running on http://localhost:\${PORT});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 92:** `logger.info(🗄 Database: PostgreSQL via Prisma );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 93:** `logger.info(🗄 Redis: ${process.env.REDIS_URL || 'redis://localhost:6379'} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 96:** `startWorkflowWorker();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 97:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 100:** `export { app, io, httpServer };`
 - *Explanation:* Executes internal logical operations for this block of code.

File: `packages/web/lib/api.ts`

Source Code

```
// =====
// FlowForge – API Client
// =====
// This is the bridge between the frontend and the backend.
// All API calls go through this client.
//
// HOW IT WORKS:
// 1. It wraps the native `fetch` API
// 2. Automatically adds the auth token to every request
// 3. Handles errors consistently
// 4. Provides typed methods for each endpoint

const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:4000';

// Get the auth token from localStorage
function getToken(): string | null {
  if (typeof window !== 'undefined') {
    return localStorage.getItem('flowforge-token');
  }
  return null;
}

// Save the auth token to localStorage
```

```

export function setToken(token: string): void {
  if (typeof window !== 'undefined') {
    localStorage.setItem('flowforge-token', token);
  }
}

// Remove the auth token (on logout)
export function removeToken(): void {
  if (typeof window !== 'undefined') {
    localStorage.removeItem('flowforge-token');
  }
}

// Generic API response type
interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: {
    message: string;
  };
}

// The main fetch wrapper
async function apiRequest<T>(<
  endpoint: string,
  options: RequestInit = {},
): Promise<ApiResponse<T>> {
  const token = getToken();

  const headers: HeadersInit = {
    'Content-Type': 'application/json',
    ...(token && { Authorization: `Bearer ${token}` }),
    ...options.headers,
  };

  try {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      ...options,
      headers,
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.error?.message || `API Error: ${response.status}`);
    }

    return data;
  } catch (error) {
    if (error instanceof Error) {
      throw error;
    }
  }
}

```

```

        throw new Error('An unexpected error occurred');
    }
}

// ---- Auth API ----

export interface LoginData {
    user: { id: string; name: string; email: string; avatarUrl: string | null };
    workspaces: Array<{ id: string; name: string; slug: string; role: string }>;
    token: string;
}

export interface RegisterData {
    user: { id: string; name: string; email: string; avatarUrl: string | null };
    workspace: { id: string; name: string; slug: string };
    token: string;
}

export const authApi = {
    login: (email: string, password: string) =>
        apiRequest<LoginData>('/api/auth/login', {
            method: 'POST',
            body: JSON.stringify({ email, password }),
        }),

    loginWithGoogle: (credential: string) =>
        apiRequest<LoginData>('/api/auth/google', {
            method: 'POST',
            body: JSON.stringify({ credential }),
        }),

    register: (name: string, email: string, password: string) =>
        apiRequest<RegisterData>('/api/auth/register', {
            method: 'POST',
            body: JSON.stringify({ name, email, password }),
        }),
};

// ---- Workflows API ----

export interface Workflow {
    id: string;
    name: string;
    description: string | null;
    status: 'DRAFT' | 'ACTIVE' | 'PAUSED' | 'ARCHIVED';
    triggerType: 'MANUAL' | 'CRON' | 'WEBHOOK';
    createdAt: string;
    updatedAt: string;
    nodesJson?: unknown[];
    edgesJson?: unknown[];
    createdBy?: { id: string; name: string; avatarUrl: string | null };
    _count?: { executions: number };
}

```

```

}

export const workflowApi = {
  list: (workspaceId: string) =>
    apiRequest<Workflow[]>(`/api/workflows?workspaceId=${workspaceId}`),

  get: (id: string) => apiRequest<Workflow>(`/api/workflows/${id}`),

  create: (data: { name: string; description?: string; workspaceId: string }) =>
    apiRequest<Workflow>(`/api/workflows`, {
      method: 'POST',
      body: JSON.stringify(data),
    }),

  update: (id: string, data: Partial<Workflow>) =>
    apiRequest<Workflow>(`/api/workflows/${id}`, {
      method: 'PUT',
      body: JSON.stringify(data),
    }),

  delete: (id: string) =>
    apiRequest<void>(`/api/workflows/${id}`, {
      method: 'DELETE',
    }),
};

// ---- Execution API ----

export interface Execution {
  id: string;
  status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED';
  triggeredBy: string;
  startedAt: string;
  completedAt: string | null;
  duration: number | null;
  errorMessage: string | null;
  workflow?: { id: string; name: string };
  nodeExecutions?: NodeExecution[];
  _count?: { nodeExecutions: number };
}

export interface NodeExecution {
  id: string;
  nodeId: string;
  nodeType: string;
  nodeName: string;
  status: 'PENDING' | 'RUNNING' | 'SUCCESS' | 'FAILED' | 'SKIPPED';
  outputData: unknown;
  errorMessage: string | null;
  startedAt: string | null;
  completedAt: string | null;
  duration: number | null;
}

```

```

}

export const executionApi = {
  execute: (workflowId: string) =>
    apiRequest<{ executionId: string; status: string }>(
      `/api/workflows/${workflowId}/execute`,
      { method: 'POST' },
    ),

  list: (workspaceId: string) =>
    apiRequest<Execution[]>(`/api/executions?workspaceId=${workspaceId}`),

  get: (id: string) => apiRequest<Execution>(`/api/executions/${id}`),
};

```

Line-by-Line Breakdown

- **Line 13:** `const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:4000';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 16:** `function getToken(): string | null {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 17:** `if (typeof window !== 'undefined') {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 18:** `return localStorage.getItem('flowforge-token');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 19:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 20:** `return null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 21:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 24:** `export function setToken(token: string): void {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 25:** `if (typeof window !== 'undefined') {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 26:** `localStorage.setItem('flowforge-token', token);`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 27:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 28:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 31:** `export function removeToken(): void {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 32:** `if (typeof window !== 'undefined') {`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 33:** `localStorage.removeItem('flowforge-token');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 34:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 35:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 38:** `interface ApiResponse<T> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 39:** `success: boolean;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 40:** `data?: T;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 41:** `error?: {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 42:** `message: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 43:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 44:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 47:** `async function apiRequest<T>(
 - Explanation: Executes internal logical operations for this block of code.`
- **Line 48:** `endpoint: string,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 49:** `options: RequestInit = {},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 50:** `): Promise<ApiResponse<T>> {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 51:** `const token = getToken();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 53:** `const headers: HeadersInit = {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 54:** `'Content-Type': 'application/json',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 55:** `...(token && { Authorization: Bearer ${token} } ),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 56:** `...options.headers,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 57:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 59:** `try {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 60:** `const response = await fetch( ${API_BASE_URL}${endpoint} , {`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 61:** `...options,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 62:** `headers,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 63:** `});`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 65:** `const data = await response.json();`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 67:** `if (!response.ok) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 68:** `throw new Error(data.error?.message || API Error: ${response.status} );`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 69:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 71:** `return data;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 72:** `} catch (error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 73:** `if (error instanceof Error) {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 74:** `throw error;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 75:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 76:** `throw new Error('An unexpected error occurred');`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 77:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 78:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 82:** `export interface LoginData {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 83:** `user: { id: string; name: string; email: string; avatarUrl: string | null };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 84:** `workspaces: Array<{ id: string; name: string; slug: string; role: string }>;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 85:** `token: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 86:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 88:** `export interface RegisterData {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 89:** `user: { id: string; name: string; email: string; avatarUrl: string | null };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 90:** `workspace: { id: string; name: string; slug: string };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 91:** `token: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 92:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 94:** `export const authApi = {`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
- **Line 95:** `login: (email: string, password: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 96:** `apiRequest<LoginData>('/api/auth/login', {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 97:** `method: 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 98:** `body: JSON.stringify({ email, password })),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 99:** `}},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 101:** `loginWithGoogle: (credential: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 102:** `apiRequest<LoginData>('/api/auth/google', {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 103:** `method: 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 104:** `body: JSON.stringify({ credential })),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 105:** `}},`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 107:** `register: (name: string, email: string, password: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 108:** `apiRequest<RegisterData>('/api/auth/register', {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 109:** `method: 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 110:** `body: JSON.stringify({ name, email, password })),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 111:** `}},`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 112:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 116:** `export interface Workflow {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 117:** `id: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 118:** `name: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 119:** `description: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 120:** `status: 'DRAFT' | 'ACTIVE' | 'PAUSED' | 'ARCHIVED';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 121:** `triggerType: 'MANUAL' | 'CRON' | 'WEBHOOK';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 122:** `createdAt: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 123:** `updatedAt: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 124:** `nodesJson?: unknown[];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 125:** `edgesJson?: unknown[];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 126:** `createdBy?: { id: string; name: string; avatarUrl: string | null };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 127:** `_count?: { executions: number };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 128:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 130:** `export const workflowApi = {`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
- **Line 131:** `list: (workspaceId: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 132:** `apiRequest<Workflow[]>( /api/workflows?workspaceId=${workspaceId} ),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 134:** `get: (id: string) => apiRequest<Workflow>( /api/workflows/${id} ),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 136:** `create: (data: { name: string; description?: string; workspaceId: string }) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 137:** `apiRequest<Workflow>('/api/workflows', {`
 - *Explanation:* Executes internal logical operations for this block of code.

- **Line 138:** `method: 'POST',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 139:** `body: JSON.stringify(data),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 140:** `}),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 142:** `update: (id: string, data: Partial<Workflow>) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 143:** `apiRequest<Workflow>( /api/workflows/${id} , {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 144:** `method: 'PUT',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 145:** `body: JSON.stringify(data),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 146:** `}),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 148:** `delete: (id: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 149:** `apiRequest<void>( /api/workflows/${id} , {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 150:** `method: 'DELETE',`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 151:** `}),`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 152:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 156:** `export interface Execution {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 157:** `id: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 158:** `status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 159:** `triggeredBy: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 160:** `startedAt: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 161:** `completedAt: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 162:** `duration: number | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 163:** `errorMessage: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 164:** `workflow?: { id: string; name: string };`

- *Explanation:* Executes internal logical operations for this block of code.
- **Line 165:** `nodeExecutions?: NodeExecution[];`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 166:** `_count?: { nodeExecutions: number };`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 167:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 169:** `export interface NodeExecution {`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 170:** `id: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 171:** `nodeId: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 172:** `nodeType: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 173:** `nodeName: string;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 174:** `status: 'PENDING' | 'RUNNING' | 'SUCCESS' | 'FAILED' | 'SKIPPED';`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 175:** `outputData: unknown;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 176:** `errorMessage: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 177:** `startedAt: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 178:** `completedAt: string | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 179:** `duration: number | null;`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 180:** `}`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 182:** `export const executionApi = {`
 - *Explanation:* Makes this specific function or variable publicly available to be imported by other files.
- **Line 183:** `execute: (workflowId: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 184:** `apiRequest<{ executionId: string; status: string }>({`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 185:** ``/api/workflows/${workflowId}/execute` ,`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 186:** `{ method: 'POST' },`
 - *Explanation:* Executes internal logical operations for this block of code.
- **Line 187:** `),`

- *Explanation:* Executes internal logical operations for this block of code.
 - **Line 189:** `list: (workspaceId: string) =>`
 - *Explanation:* Executes internal logical operations for this block of code.
 - **Line 190:** `apiRequest<Execution[]>( /api/executions?workspaceId=${workspaceId} ),`
 - *Explanation:* Executes internal logical operations for this block of code.
 - **Line 192:** `get: (id: string) => apiRequest<Execution>( /api/executions/${id} ),`
 - *Explanation:* Executes internal logical operations for this block of code.
 - **Line 193:** `};`
 - *Explanation:* Executes internal logical operations for this block of code.
-